

Arab Academy for Science, Technology and Maritime Transport

College of Engineering and Technology

Computer Engineering Department

B.Sc. Final Year Project

Predictive Maintenance for OBD-II Engine Faults

Presented by:

Omar Hossam Eldin Mohamed Hafez	Reg. No. 20101737
Mohamed Sameh Mohamed Mohamed El Bassuni	Reg. No. 18100896
Mohamed Mohamed Talaat Abokhatwa	Reg. No. 12103172
Ahmed Abdelaal Saad	Reg. No. 16101027
Adam Mohamed Ibrahim	Reg. No. 20101978

Supervised by:

Yahia Elderiny

Rania Kadry

July — 2026

Declaration

I hereby certify that this report, which I am now submitting for evaluation on the study program leading to the award of Bachelor of Science in Computer Engineering, is all my own work and contains no plagiarism. I comply with the following terms when submitting this report:

Any text, diagrams or other material copied from other sources (including, but not limited to, books, journals, and the internet) have been clearly acknowledged and cited, followed by the reference number used, either in the text or in a footnote/endnote. The details of the used references that are listed at the end of the report conform to the referencing style dictated by the final year project template and are, to my knowledge, accurate and complete.

I have read the sections on referencing and plagiarism in the final year project template. I understand that plagiarism can lead to a reduced or fail grade, in serious cases, for the Graduation Project course.

Student name: Omar Hossam Eldin Mohamed Hafez
Registration number: 20101737
Signed: _____
Date: _____

Student name: Mohamed Sameh Mohamed Mohamed El Bassuni
Registration number: 18100896
Signed: _____
Date: _____

Student name: Mohamed Mohamed Talaat Abokhatwa
Registration number: 12103172
Signed: _____
Date: _____

Student name: Ahmed Abdelaal Saad
Registration number: 16101027
Signed: _____
Date: _____

Student name: Adam Mohamed Ibrahim
Registration number: 20101978
Signed: _____
Date: _____

Senior Project Summary Report

Arab Academy for Science, Technology and Maritime Transport
College of Engineering and Technology — Department of Computer Engineering
Academic Year: 2025/2026 Semester: 10

Project title	Predictive Maintenance for OBD-II Engine Faults
Supervisor(s)	Yahia Elderiny; Rania Kadry
Team members	Omar Hossam Eldin Mohamed Hafez — 20101737 Mohamed Sameh Mohamed Mohamed El Bassuni — 18100896 Mohamed Mohamed Talaat Abokhatwa — 12103172 Ahmed Abdelaal Saad — 16101027 Adam Mohamed Ibrahim — 20101978
Project deliverables	A physics-constrained fault injection engine for OBD-II time series; a six-class XGBoost fault classifier with SHAP explanations; a PID forecaster and one-class anomaly detector; an Android data-collection app (OBD-LOG); a Streamlit dashboard with an end-of-read health report; and a documented protocol for future real-fault data collection.
Description	The system reads live engine sensor data through the standard OBD-II port at 1 Hz, converts 60-second windows into 83 vehicle-agnostic features, and classifies the engine state into healthy, cold-start, or one of four developing fault classes (vacuum leak, fuel delivery, coolant sensor, throttle sensor). Because no public real-fault OBD-II dataset exists at this sample rate, training data is produced by injecting physically plausible fault signatures into 84.5 hours of real healthy driving data. The pipeline transfers across vehicles by z-scoring every feature against the target car's own healthy baseline, and it reports "Untested" for any fault whose required sensor a given car does not expose.

Engineering standards	SAE J1979 (OBD-II diagnostic services and PID definitions); ISO 15765-4 (CAN transport for diagnostics); ELM327 AT command set for adapter control; OBD-II physical ranges used as hard clamps in the injection engine (fuel trim ±25 %, timing advance ±64°).
Design constraints	1 Hz sampling ceiling of consumer ELM327 adapters (measured 0.37–1.10 Hz depending on PID count); Bluetooth Classic SPP throughput; per-vehicle PID availability (the test Yaris exposes no MAP sensor); no labeled real-fault data available anywhere at 1 Hz; laptop-only inference (no cloud dependency for the demo); all injected values clamped to engine physics.
Project impact	Earlier warning of developing engine faults than the check-engine light, using hardware the average driver can afford (a phone plus a $10 adapter); a reusable, physics-grounded methodology for synthesizing labeled fault data where none exists; an honest evaluation framework that reports what it cannot test instead of fabricating scores.
Team organization	Omar Hossam Eldin Mohamed Hafez — data pipeline and fault injection. Mohamed Sameh Mohamed Mohamed El Bassuni — Android application and vehicle testing. Mohamed Mohamed Talaat Abokhatwa — feature engineering and classifier. Ahmed Abdelaal Saad — forecaster, anomaly detection, evaluation. Adam Mohamed Ibrahim — dashboard, integration, documentation.
Ethics / safety	No physical fault was induced on any test vehicle. All driving data was collected on public roads with the driver's consent, at legal speeds, with the phone mounted and operated by a passenger. Model outputs are advisory only; a qualified mechanic must confirm any suspected fault before repair decisions. The synthetic origin of all fault labels is disclosed in every result table.

Main Supervisor Signature: _____

Students Contribution

Predictive Maintenance for OBD-II Engine Faults

Submitted by: Omar Hossam Eldin Mohamed Hafez, Mohamed Sameh Mohamed Mohamed El Bassuni, Mohamed Mohamed Talaat Abokhatwa, Ahmed Abdelaal Saad, Adam Mohamed Ibrahim

Chapter	**Title**	**Contributors**
1	Introduction	Omar Hossam Eldin Mohamed Hafez Adam Mohamed Ibrahim
2	Literature Survey and Related Work	Mohamed Sameh Mohamed Mohamed El Bassuni Mohamed Mohamed Talaat Abokhatwa
3	The Proposed System	Omar Hossam Eldin Mohamed Hafez Mohamed Sameh Mohamed Mohamed El Bassuni Mohamed Mohamed Talaat Abokhatwa
4	Experimental Results and Evaluation	Ahmed Abdelaal Saad Adam Mohamed Ibrahim
5	Conclusion and Future Work	Ahmed Abdelaal Saad Adam Mohamed Ibrahim

Acknowledgment

We wish to express our sincere gratitude to all those who gave us the opportunity to make tangible progress in this project.

First and foremost, we thank God Almighty for giving us the strength, knowledge, ability and opportunity to undertake this work and to persevere and complete it satisfactorily.

Special appreciation for the supervision, advice and continuous encouragement goes to our advisors, Yahia Elderiny and Rania Kadry.

We would also like to thank the Arab Academy for Science, Technology and Maritime Transport for the facilities and support that made this project possible, and Eron J. Maranhão for publishing the carOBD dataset that our training pipeline is built on.

Last but not least, our heartfelt gratitude goes to our families for their support throughout the year, and to the long-suffering 2015 Toyota Yaris that idled through every data-collection session without complaint.

Abstract

Modern cars continuously report engine sensor data through the standardized OBD-II port, yet that data is used almost exclusively after a fault has already tripped a diagnostic trouble code. This project builds a predictive maintenance system that watches the sensor stream itself and flags a developing fault before the check-engine light fires. The system reads 14 engine parameters at 1 Hz, summarizes each 60-second window into 83 vehicle-agnostic features, and classifies the window into healthy, cold-start, or one of four fault classes: intake vacuum leak, fuel delivery fault, coolant temperature sensor fault, and throttle position sensor fault.

No public OBD-II dataset with labeled real faults exists at this sample rate, so we trained on synthetic faults injected into 84.5 hours of real healthy driving data from the public carOBD corpus — 129 sessions that we recovered from an upstream CSV parsing defect which had previously made 120 of them appear corrupt. The injection engine is physics-constrained: every written value respects engine causality, from the 1°C-per-second thermal inertia of coolant to the short-term-to-long-term fuel trim handoff of a real adaptive ECU. An XGBoost classifier reaches a leave-one-family-out macro-F1 of 0.977 (worst family 0.958), and we report these numbers explicitly as synthetic self-consistency scores, not as proof of real-fault detection. On a real, healthy 2015 Toyota Yaris the deployed pipeline correctly declared the air-system class Untested (the car has no MAP sensor) and refused to issue a verdict on insufficient evaluable data — behavior we argue is an engineering contribution in itself, because an earlier version of the same pipeline confidently reported a 97 % air-system fault on the same healthy car. The system ships with an Android data-collection app, a live Streamlit dashboard, and a documented protocol for the real-fault recordings that would convert the synthetic validation into field validation.

Contents

Declaration ii

Senior Project Summary Report iii

Students Contribution vi

Acknowledgment vii

Abstract viii

List of Figures xi

List of Tables xii

List of Acronyms xiii

1 Introduction 1

1.1 Overview . . . 1

1.2 Problem Statement . . . 1

1.3 Motivation and Application . . . 2

1.4 Challenges . . . 3

1.5 Objectives . . . 4

1.6 Ethics . . . 4

1.7 Report Layout . . . 5

2 Literature Survey and Related Work 6

2.1 Overview . . . 6

2.2 Related Work . . . 6

2.2.1 Diagnostics on the OBD-II interface . . . 6

2.2.2 Machine learning for automotive predictive maintenance . . . 7

2.2.3 Models and attribution . . . 7

2.3 Gap Analysis . . . 8

2.4 How This Project Fills the Gap . . . 9

3 The Proposed System **10**

3.1 System Architecture 10

3.2 Data Collection — the OBD-LOG Android App 11

3.3 Dataset — carOBD 12

3.3.1 The trailing-comma discovery 12

3.3.2 Working PID set 13

3.4 Fault Injection Engine 14

3.5 Feature Engineering 15

3.6 Model Development 16

3.7 Dashboard 18

4 Experimental Results and Evaluation **19**

4.1 Evaluation Methodology 19

4.2 Classification Results 20

4.3 Data Exploration Results 22

4.4 Real-Vehicle Testing 25

4.5 Forecaster Results 26

4.6 Discussion 27

5 Conclusion and Future Work **29**

5.1 Conclusions 29

5.2 Limitations 30

5.3 Future Work 31

A Key Source Code Listings — Analysis Pipeline **35**

B Key Source Code Listings — OBD-LOG Android App **54**

C Full Per-Class Metrics Tables **59**

List of Figures

1 System architecture. The top row is the acquisition path on the vehicle; the middle row is the training pipeline built on the carOBD corpus; the bottom row is the inference path that scores recordings and drives the dashboard. . 10

2 The column-shift defect seen through one PID. Left: under a bare `read_csv`, the TIMING_ADVANCE column contains a large population far outside the ±64° OBD-II bound (dashed lines), peaking at 776.6° — catalyst temperatures wearing the wrong column name. Right: with `index_col=False` the same files yield −24° to +35°, entirely inside the physical range. Note the logarithmic count axis. 13

3 Fixed-holdout confusion matrix for the six-class XGBoost classifier (sessions drive1 + live12; macro-F1 = 0.962). 22

4 Corpus composition on both axes. By file count the idle family dominates (47 of 129); by 60-second windows, cruise dominates (62.9 %) because driving sessions are far longer than idle ones. 23

5 Distributions of the five key fault-bearing PIDs across all 129 files. Fuel trims are roughly symmetric and slightly rich-shifted with visible OBD-II quantization; coolant is bimodal (cold-start shoulder plus a warm-operation peak near 88–90°C); MAP is multimodal from the idle/driving mixture; timing sits entirely inside its physical envelope. 23

6 Correlation heatmap of the 14 working PIDs over the full corpus. 24

7 Bivariate structure, colored by a row-level regime proxy. Left: MAP vs RPM separates the idle cluster from the driving cloud — the region the air-system fault perturbs. Center: RPM vs speed resolves into discrete gear bands, a drivetrain sanity check that a residual column shift would scramble. Right: timing advance climbs and stabilizes as coolant warms — the coupling the coolant-sensor fault exploits. 25

List of Tables

2 Related work along the two axes that matter for our problem: fault-label realism and data interface. No surveyed work combines verified faults with the 1 Hz consumer OBD-II interface. 8

3 Measured poll rate versus polled PID count (2015 Toyota Yaris, ELM327 Bluetooth clone, parked idle). 11

4 Classifier results across evaluation paths. All numbers measure recovery of synthetically injected faults (self-consistency), not real-fault detection. . . . 20

5 Per-class F1 for the worst LOFO fold (live family held out; 39 sessions, 30,336 test windows). 20

6 Real-vehicle scoring outcomes on the healthy 2015 Toyota Yaris, with the Untested contract applied. Raw label counts are the classifier's per-window outputs before suppression. 26

7 PID forecaster mean absolute error (z-units, 60 s horizon, held-out sessions) versus the persistence baseline. 27

8 XGBoost fixed-holdout (drive1 + live12) per-class metrics. Macro-F1 = 0.962. 59

9 Random Forest baseline, fixed holdout, earlier five-class task. Macro-F1 = 1.000 — reported as a memorization warning, not a result. 59

10 LOFO cross-validation: per-class F1 for every held-out family. Mean macro-F1 = 0.977, $\sigma = 0.012$. 60

11 LOSO cross-validation summary over all 129 single-session folds. 60

12 Withheld-configuration experiment: train on injector configuration A, evaluate on configuration B. 60

List of Acronyms

OBD-II On-Board Diagnostics, second generation — the standardized vehicle diagnostic interface mandated on passenger cars since 1996

PID Parameter ID — one addressable sensor or computed value in the OBD-II protocol (e.g. 0x0C = engine RPM)

ELM327 The de-facto standard microcontroller that bridges the OBD-II bus to a serial (Bluetooth/USB) link using AT commands

ECU Engine Control Unit — the embedded computer that manages fueling, ignition and emissions

MAF Mass Air Flow sensor — measures intake airflow directly

MAP Manifold Absolute Pressure sensor — measures intake manifold pressure; the airflow reference on speed-density engines

XGBoost eXtreme Gradient Boosting — a gradient-boosted decision tree library

SHAP SHapley Additive exPlanations — a game-theoretic method for attributing a model prediction to its input features

ML Machine Learning

LOFO Leave-One-Family-Out cross-validation (holding out an entire session family, e.g. all idle recordings)

LOSO Leave-One-Session-Out cross-validation

LTFT Long-Term Fuel Trim — the ECU's learned, persistent fuel correction, in percent

STFT Short-Term Fuel Trim — the ECU's fast, oscillating fuel correction, in percent

DTC Diagnostic Trouble Code — the standardized fault code that turns on the check-engine light

API Application Programming Interface

CSV Comma-Separated Values file format

RPM Revolutions Per Minute (engine speed)

IAT Intake Air Temperature

TPS Throttle Position Sensor

APK Android Package — the installable Android application format

BOM Byte Order Mark — a Unicode signature at the start of a text file

Kafka Apache Kafka — a distributed event-streaming platform

REST REpresentational State Transfer — an HTTP API architectural style

1 Introduction

1.1 Overview

Every passenger car built since the mid-1990s carries a diagnostic socket under its dashboard. Through that socket — the OBD-II port — the engine control unit (ECU) will answer questions about itself once per second: engine speed, coolant temperature, how much fuel correction it is currently applying, how far the throttle plate is open, and dozens more. A ten-dollar ELM327 Bluetooth adapter turns any phone into a reader for this stream. The data is there, in every car, all the time.

What the industry does with that stream is mostly reactive. The ECU compares individual sensors against fixed thresholds, and when one is crossed for long enough, it stores a diagnostic trouble code (DTC) and lights the check-engine lamp. By that point the fault is established. The slow drift that preceded it — a fuel trim creeping upward for weeks as an injector clogs, an idle speed rising a little as a vacuum line cracks — is visible in the sensor stream long before any threshold trips, but nothing in the standard diagnostic chain looks for it.

We built a system that looks for it. The system ingests the 1 Hz OBD-II stream, summarizes each 60-second window of driving into a feature vector, and classifies the engine's state into healthy, cold-start, or one of four developing fault classes. It runs on a laptop, takes its data from either a recorded CSV file or a live Bluetooth adapter, explains each prediction in terms of the sensors that drove it, and ends every session with a plain verdict about the engine's health. Alongside the analysis pipeline we built an Android application, OBD-LOG, that performs the vehicle-side data collection on real cars.

One fact shapes the entire project, and we state it here rather than bury it: there is no public dataset of real, labeled engine faults recorded through OBD-II at 1 Hz. Nobody has published one. Our models are therefore trained on *synthetic* faults — physically grounded modifications injected into 84.5 hours of real healthy driving data. Chapter 4 quantifies exactly what that means for how the results should be read.

1.2 Problem Statement

Given a stream of standard OBD-II sensor readings sampled at approximately 1 Hz from an unmodified passenger car, decide — within about a minute of data — whether the engine is behaving normally, warming up from a cold start, or exhibiting the early signature of one of four common fault classes: an intake vacuum leak, a fuel delivery restriction, a stuck coolant temperature sensor, or a worn throttle position sensor. The decision must come with an explanation a mechanic can check, must transfer to vehicles the model has

never seen, and must refuse to answer rather than guess when the target vehicle does not expose the sensors a given fault requires.

Three properties of the setting make this harder than a textbook classification exercise. The data rate is low: at 1 Hz, combustion-level events are invisible, and only slow aggregate signatures survive. The labels do not exist: healthy public data is plentiful, but labeled fault data at this rate is not, so the training signal has to be synthesized without violating engine physics. And the deployment target varies: two cars expose different PID sets, idle at different speeds, and trim fuel around different baselines, so a model fitted to one car's raw values fails on the next car for reasons that have nothing to do with faults.

1.3 Motivation and Application

The gap between "data available" and "warning delivered" is the motivation. A drivetrain fault that is caught while it is still a drifting sensor value costs a hose clamp or a sensor swap; the same fault caught at the DTC stage may have been degrading fuel economy for months, and caught at the breakdown stage it costs a tow. Fleet operators already pay for telematics systems that promise early warning, but those systems are proprietary, and the published research behind them almost never operates on the 1 Hz consumer-grade interface that ordinary vehicles actually expose.

Consider the concrete case of a clogging fuel injector. Early, it shows only as a long-term fuel trim that has crept from its normal few percent to eight or nine, correcting for a mixture that is running slightly lean. The car drives fine; the driver notices nothing; no code is stored. Weeks later the trim saturates at its limit, the mixture goes lean under load, and now there is a misfire code, a rough idle, and possibly a damaged catalytic converter downstream. The whole trajectory was visible in one PID from the first week. A tool that flagged the drift at week one turns a converter and an injector into just an injector, and turns a roadside stall into a scheduled appointment. That is the value proposition in a single fault, and three of our four fault classes tell structurally similar stories.

The intended applications are concrete. A workshop can replay a customer's recorded drive and get a per-fault verdict with sensor-level evidence. A student or hobbyist can point the dashboard at a live car and watch the classifier, the fuel trims, and the SHAP explanation update together. A fleet, with the future-work pipeline of Section 5.3, could stream many vehicles into the same scoring backend. In every case the hardware cost is a phone and an ELM327 adapter — perhaps fifteen dollars of equipment reading data the car already computes and publishes for free.

There is also a methodological application that outlives this particular system. The fault injection engine of Section 3.4 is a worked answer to a question many practitioners

face: how do you train a detector when the failure data does not exist? Our answer — inject faults into real healthy data under hard physical constraints, document every parameter, and measure how much the scores depend on the injector's own settings — is reusable well beyond these four fault classes.

1.4 Challenges

The project's difficulties were not where we first expected them. Put simply, the models were the easy part; the data was not.

- **A corrupted-looking corpus.** Our first audit of the public carOBD dataset concluded that 120 of its 129 files contained physically impossible sensor values and had to be discarded. The real cause turned out to be a single-character CSV parsing defect — a trailing comma that silently shifted every column one position left. Diagnosing this took longer than training any model in this report, and Section 3.3 documents it in full.
- **Synthesizing faults without lying about physics.** A naive fault injector produces textbook signatures that real ECUs never emit. A vacuum leak on a speed-density engine, for example, does *not* produce the large fuel-trim swing the textbooks describe, because the MAP sensor sees the leak directly and the ECU compensates. Getting each of the four fault models to match documented ECU behavior forced us to discard and rewrite our first implementations.
- **Evaluation that does not flatter itself.** Windows cut from the same drive overlap by up to 50 of their 60 rows. Any split that mixes one session's windows across train and test leaks information and inflates F1 by 10–20 points. We had to lock session-level splitting in as a tested invariant, and we had to accept that our headline numbers measure self-consistency with our own injector, not field detection.
- **Cross-vehicle reality.** The training vehicle (Toyota Etios) is a speed-density car with a MAP sensor; our test vehicle (Toyota Yaris) is MAF-based and reports no MAP at all. The first deployment on the healthy Yaris produced a confident, entirely wrong fault alarm, and fixing that failure properly reshaped the system's reporting layer.
- **Throughput of consumer hardware.** A real ELM327 clone polls 7 PIDs at 1.10 Hz but 21 PIDs at only 0.37 Hz. Choosing which PIDs are worth their cost in sample rate became a design decision with measured trade-offs, not a footnote.

None of these five were solved by writing more machine learning code. Each one was a data problem, a physics problem, or a hardware problem first, and the model only got

to be as good as the data underneath it allowed. That ordering — fix the data, then fit the model — turned out to be the project's actual methodology, even though it was not the one we set out with in Week 1.

1.5 Objectives

1. Recover and validate the full 129-session carOBD corpus, with a loud, tested guard against the column-shift defect that had hidden 93 % of it.
2. Build a fault injection engine for four fault classes, in ramp and step modes, in which every written value is clamped to engine physics and every injection is reproducible from stored parameters.
3. Design a vehicle-agnostic feature representation — 83 features per 60-second window, z-scored against the source vehicle's own healthy baseline — that lets a model trained on one car run on another.
4. Train and evaluate a six-class XGBoost classifier under session-level and family-level cross-validation, reporting synthetic self-consistency scores as exactly that.
5. Complement the classifier with a per-PID forecaster and a one-class anomaly detector, and report honestly where they fail (long-term fuel trim forecasting fails).
6. Build the OBD-LOG Android app for real-vehicle data collection over Bluetooth, with tiered PID selection driven by measured poll rates.
7. Deploy the full pipeline against real recordings from an unseen vehicle, and design the reporting layer so that missing sensors produce "Untested" verdicts rather than fabricated scores.
8. Deliver a live dashboard that shows the classification, its SHAP explanation, physics-based severity, and an end-of-read health report.

1.6 Ethics

No physical fault was induced on any vehicle at any point in this project. Every fault our models were trained on is synthetic, and every real recording we scored came from a healthy, unmodified car. We considered inducing a real vacuum leak under supervision and decided against it for this phase; the documented collection protocol in the repository specifies how to do it safely if a future team takes that step, with a mechanic present and abort conditions defined in advance.

All driving data we collected ourselves was recorded on public roads with the driver's knowledge and consent, at legal speeds, with the phone mounted and the app operated by a passenger. The carOBD corpus we trained on is public, published by its author for research use, and credited in Chapter 2 and the references.

The system's outputs are advisory. A classifier label on a dashboard is not a diagnosis, and the report says so on its face: every end-of-read verdict carries its caveats in the rendered output, including the synthetic origin of the training data and the conditions under which the verdict is unreliable. A qualified mechanic must confirm any suspected fault before a repair decision is made. We consider the refusal behaviors — "Untested" when a required sensor is absent, "insufficient data" when too few windows are evaluable — to be ethics features as much as engineering ones, because the alternative is a tool that manufactures confident false alarms, and we watched an early version of our own system do exactly that.

1.7 Report Layout

Chapter 2 surveys prior work on OBD-II diagnostics and machine learning for predictive maintenance, and identifies the gap this project fills. Chapter 3 describes the proposed system: architecture, the Android data-collection app, the dataset and its recovery, the fault injection engine, feature engineering, the models, and the dashboard. Chapter 4 presents the evaluation methodology and results, including the real-vehicle deployment and the failures we kept. Chapter 5 concludes with limitations, stated plainly, and a prioritized future-work list. The appendices contain the key source code listings from both repositories and the full per-class metric tables.

2 Literature Survey and Related Work

2.1 Overview

Work relevant to this project falls into three strands: diagnostics built directly on the OBD-II interface, machine learning for automotive predictive maintenance in general, and the specific model family (gradient-boosted trees with attribution methods) that we deploy. Reading across the three strands, a consistent pattern emerges: the studies with realistic fault data do not use the consumer OBD-II interface, and the studies on the consumer interface do not have realistic fault data. Our project sits precisely in that intersection, which is why the survey below pays as much attention to each paper's *data source* as to its model.

2.2 Related Work

2.2.1 Diagnostics on the OBD-II interface

Rimpas et al. [1] logged OBD-II parameters — fuel trims, coolant temperature, ignition timing, and airflow among them — to monitor vehicle operation and fuel consumption, and discussed how deviations in individual PIDs indicate specific degradations. Their work confirms the diagnostic value of exactly the PIDs our system builds features on, but the analysis is per-parameter and threshold-style. No classifier is trained, and no labeled faults are used; the interpretation of a drifting trim value is left to the reading engineer.

Shafi et al. [2] built a remote health-monitoring system with an OBD-II dongle streaming to a phone and applied several classifiers (decision trees, SVM, k-NN) to detect anomalies in four vehicle subsystems. Their fault examples, however, come from threshold rules applied to the same stream the classifiers read — the labels are derived from the data rather than from independently verified physical faults. The system demonstrates the plumbing of phone-based OBD-II ML but not validated fault detection.

The carOBD dataset and its accompanying thesis by Maranhão [3] provide the raw material our project trains on: 129 sessions of 1 Hz OBD-II logging from a single Toyota Etios, covering idle, commute, campus, highway, and long-trip driving. The dataset contains healthy driving only. To our knowledge no public OBD-II corpus at this rate contains verified, labeled faults; we searched for one and did not find it, and this absence is the central constraint the project designs around.

That absence is structural, not an oversight anyone will quickly correct. Collecting labeled fault data at scale means deliberately breaking cars, or waiting for a fleet's worth of them to break naturally while logging continuously, and then joining the logs to verified

repair records. The first is expensive and, done carelessly, dangerous; the second requires fleet access and years of patience. Whoever pays for either has a commercial reason to keep the result private. The public-data vacuum that Theissler et al. describe at the field level is, in other words, structural, and a graduation project cannot fill it by trying harder. It can only decide how to work honestly inside it, which is the decision Section 3.4 documents.

2.2.2 Machine learning for automotive predictive maintenance

Theissler et al. [4] survey machine learning for predictive maintenance across the automotive industry and catalogue the field's recurring obstacles: scarce labeled failure data, class imbalance, and the gap between lab results and fleet deployment. They note explicitly that run-to-failure data for vehicles is rarely public because it is expensive and commercially sensitive — the same wall we hit, met in their survey as a field-wide condition rather than a local complaint.

Prytz et al. [5] predicted compressor repairs in a commercial truck fleet from logged vehicle data and workshop maintenance records. Their labels are real — actual repairs — but the data source is an OEM fleet telematics database with service-record joins, not the standardized consumer port, and the features are aggregated per vehicle-visit rather than streaming windows. The study demonstrates that drift-before-failure is learnable in principle, at fleet scale, with access no individual owner has.

Theissler [6] addressed the known/unknown fault problem with ensemble anomaly detection over in-vehicle recordings, arguing that a detector must handle fault types absent from training. Our two-pillar design — a supervised classifier for the four modeled faults plus a one-class IsolationForest [7] trained on healthy windows only — is a direct application of that argument at consumer-hardware scale.

Vergara et al. [8] published EngineFaultDB, a labeled engine fault dataset with baseline classification results. The faults are real, but they were produced on a laboratory engine test bench with instrumentation beyond the OBD-II PID set, sampled under controlled load profiles. We considered benchmarking against it and declined: a bench dynamometer corpus and a 1 Hz road-driving corpus differ in sensors, dynamics, and noise floor to the point where a comparison table would mislead more than inform.

2.2.3 Models and attribution

Gradient-boosted decision trees remain the strongest general-purpose family for tabular sensor features, and XGBoost [9] is its standard implementation; random forests [10] provide the natural bagging baseline. For attribution we use SHAP [11], whose TreeExplainer computes exact Shapley values for tree ensembles — on a workshop floor, "LTFT pushed

this prediction toward fuel-system by 0.43" is an explanation a mechanic can act on. The protocol layer underneath the whole pipeline is standardized by SAE J1979 [12] and ISO 15765-4 [13], and our tooling builds on the python-obd library [14], Streamlit [15], and FastAPI [16].

Table 2 lays the surveyed work along the two axes that decide whether a result transfers to our setting: the realism of its fault labels and the interface its data came through. The pattern is stark — every row has one or the other, never both.

Table 2: Related work along the two axes that matter for our problem: fault-label realism and data interface. No surveyed work combines verified faults with the 1 Hz consumer OBD-II interface.

Work	Fault labels	Data source
Rimpas et al. [1]	None (per-PID thresholds)	Consumer OBD-II
Shafi et al. [2]	Derived from the stream	Consumer OBD-II
Prytz et al. [5]	Real (repair records)	OEM fleet telematics
Vergara et al. [8]	Real (induced)	Bench dynamometer
Theissler [6]	Mixed known/unknown	In-vehicle recordings
This work	Synthetic, physics-constrained	Consumer OBD-II, 1 Hz

2.3 Gap Analysis

Laying the surveyed work side by side exposes four gaps.

Realistic faults and consumer hardware do not co-occur. Bench studies such as EngineFaultDB [8] have verified faults but non-OBD instrumentation and dynamometer conditions. Fleet studies such as Prytz et al. [5] have real repair labels but proprietary telematics access. OBD-II studies such as Shafi et al. [2] run on the consumer port but lack independently verified fault labels. No surveyed work trains and evaluates on the 1 Hz consumer interface with trustworthy fault ground truth — because the data to do so does not publicly exist.

Single-vehicle scope is the norm, and transfer is untested. The carOBD corpus is one Etios; EngineFaultDB is one bench engine; most OBD-II papers instrument one car. Almost none of the surveyed systems state what happens when the trained model meets a different vehicle — different idle speed, different trim baseline, different PID set. In our experience that encounter is where deployments actually fail.

Missing sensors are not handled honestly. Every surveyed system assumes its input channels exist. But PID support varies car to car: our own test vehicle exposes no MAP sensor, the primary channel for one of our four fault classes. No prior system we

found defines what it reports when the evidence for a fault class is physically absent.

Synthetic injection, where used, is not physics-audited. Adding offsets or noise to sensor channels is common in anomaly detection research, but the surveyed automotive works do not constrain injected values to ECU causality — fuel trims that lead instead of react, coolant that jumps faster than thermal mass allows, MAP exceeding barometric pressure. Models trained on physically impossible signatures learn artifacts that no real fault will ever reproduce.

2.4 How This Project Fills the Gap

Against the first gap, we accept the absence of real-fault data as a design input rather than pretending otherwise: we train on physics-constrained synthetic faults injected into real 1 Hz driving data, we measure how much the scores depend on the injector's own coefficients (the withheld-parameter experiment of Section 4.1), and we label every reported number as synthetic self-consistency. The deliverable is a detection algorithm plus a collection protocol, and the report never claims more.

Against the second, every feature is z-scored against the source vehicle's own healthy baseline (Section 3.5), so the model reasons in "standard deviations from this car's normal" rather than raw units, and the deployed pipeline re-fits that baseline per vehicle. Our evaluation splits by session and by session *family*, the closest available proxy for context transfer within a single-vehicle corpus.

Against the third, the pipeline carries an explicit evaluability contract: each fault class declares the PIDs its primary signature requires, and a vehicle that lacks them gets "Untested" for that class instead of a score (Section 3.7). Chapter 4 shows this contract working on a real MAF-based car.

Against the fourth, the injection engine enforces hard physical clamps and documented ECU response dynamics — including the speed-density vacuum-leak behavior and the STFT-to-LTFT handoff — with every injection reproducible from stored parameters (Section 3.4). The physics arguments are written into the source alongside the code they constrain.

3 The Proposed System

3.1 System Architecture

Figure 1 shows the system end to end. On the vehicle side, an ELM327 Bluetooth adapter plugs into the OBD-II port and speaks to our Android app, OBD-LOG, over a Bluetooth Classic serial link. The app polls the ECU, decodes each PID reply, and writes timestamped CSV files. Those files cross to the analysis side, where an adapter layer renames columns and resamples to a strict 1 Hz grid, and an inference engine scores each 60-second window. A Streamlit dashboard renders the live state and, at the end of a recording, an overall health report.

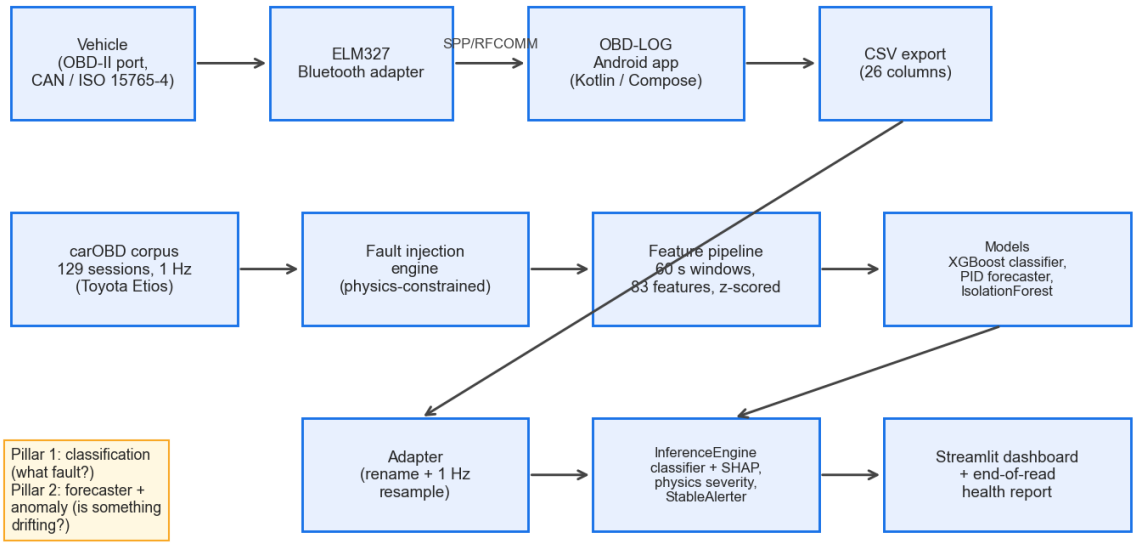


Figure 1: System architecture. The top row is the acquisition path on the vehicle; the middle row is the training pipeline built on the carOBD corpus; the bottom row is the inference path that scores recordings and drives the dashboard.

The analysis side is deliberately organized as two pillars that answer two different questions. Pillar 1 is the supervised six-class XGBoost classifier: *if* something is wrong, *which* of the modeled faults does this window look like? Pillar 2 is the pair of unsupervised safety nets — a per-PID forecaster whose prediction residuals flag drift, and a one-class IsolationForest that scores how far the current window sits from the healthy manifold. Pillar 2 exists because Pillar 1 can only name faults it was trained on; a real engine is under no obligation to fail in one of our four ways.

Both pillars consume the same feature pipeline and the same per-vehicle baseline normalization, so a single calibration step (recording a few minutes of healthy driving)

adapts the whole stack to a new car. Everything runs offline on a laptop; the live demo needs no network connection.

3.2 Data Collection — the OBD-LOG Android App

Public data covers the training side, but deploying against real cars required our own acquisition tool, so we wrote one. OBD-LOG is a native Android app in Kotlin with a Jetpack Compose / Material 3 interface. It connects to any ELM327-compatible adapter over Bluetooth Classic (SPP/RFCOMM), initializes it with the standard AT sequence (reset, echo off, linefeeds off, spaces off, automatic protocol detection), asks the ECU which PIDs it actually supports via the 0100/0120/0140 support bitmasks, and then polls the supported set in a round-robin loop, appending one CSV row per sweep. The full logger and PID-table sources are reproduced in Appendix B. Our test vehicle throughout was a 2015 Toyota Yaris, which talks ISO 15765-4 CAN [13].

The app's central design decision is a three-tier PID list, because polling is serial: every extra PID costs sample rate for all of them. Tier 1 ("Core") carries the nine highest fault-signal channels — RPM, load, MAP, MAF, coolant, intake air temperature, both fuel trims, and timing advance. Tier 2 ("Extended") adds context: speed, throttle, commanded equivalence ratio, module voltage, barometric pressure. Tier 3 ("Full") adds the remaining niche channels through to catalyst temperatures, kept mainly for throughput testing. We measured the trade-off directly on the Yaris at parked idle, and Table 3 shows why the tiers exist: tripling the PID count costs two-thirds of the sample rate.

Table 3: Measured poll rate versus polled PID count (2015 Toyota Yaris, ELM327 Bluetooth clone, parked idle).

Capture mode	PIDs polled	Measured rate	Rows in test capture
Core	7	1.10 Hz	124
Extended	12	0.64 Hz	121
Full	21	0.37 Hz	65

The three captures in Table 3 were all successful end-to-end runs: connect, init, detect, log, export. They also produced the project's single most consequential hardware finding: the Yaris returned `NO DATA` for PID 010B on every request. The car has no MAP sensor — it is a MAF-based engine — so the `INTAKE_MANIFOLD_PRESSURE` column is empty in every capture. We added MAF (PID 0110) to the Core tier in response, and Section 3.7 describes how the analysis side was redesigned to survive exactly this kind of per-vehicle sensor absence.

Each CSV carries 26 columns: `timestamp_ms` (epoch milliseconds), `datetime` (human-readable), and all 24 defined PIDs in fixed order. Unpolled or unsupported PIDs are written as empty cells rather than dropped, so every capture has an identical schema regardless of mode — a property the downstream adapter relies on. The interface is four screens behind a navigation drawer: *Dashboard* (connection control, live values, start/stop logging), *Logs* (recorded files with share/export), *Diagnostics* (the raw ELM327 init log and support-detection output, which proved invaluable against clone adapters), and *About*.

One improvement is conspicuously absent and deliberately listed as future work: a session-label text field written into the CSV, so a recording can declare itself "baseline" or "induced fault X" without an external notebook. It is a one-line UI change, and it is exactly what the real-fault collection protocol will need.

3.3 Dataset — carOBD

Training data comes from carOBD [3], a public corpus of 1 Hz OBD-II logs from a single Toyota Etios 2014 recorded through a Carloop (ELM327-compatible) device across 27 PIDs. The corpus spans five session families: parked idle (`idle`, 47 files), a recurring work–home commute (`live`, 39), highway-style driving (`drive`, 13), long trips (`long`, 12), and low-speed university-campus driving (`ufpe`, 18). After the parsing repair described next, the working corpus is 129 files, 304,299 rows, roughly 84.5 hours of driving.

3.3.1 The trailing-comma discovery

Our Week-1 audit checked every file against physical bounds for four signature PIDs and produced an alarming result: 120 of 129 files carried timing-advance values between 30° and 776°, where the OBD-II encoding cannot exceed ±64° [12]. We initially concluded the files used inconsistent logger firmware and cut the working set to the 9 clean files — about 4.4 hours of data — which is painfully little for a six-class problem.

The real cause was one character per line. The affected files end every data row with a trailing comma, so each row parses to 28 fields against 27 header names. Given that mismatch, `pandas.read_csv` silently resolves the difference by promoting the first column to the row index — and thereby shifts *every* column one position left. Coolant temperature reads fuel-trim values. Timing advance reads catalyst temperature, which is exactly why we saw "timing" of 300–776°: those are plausible catalyst temperatures in °C. The data was never corrupt. The parse was.

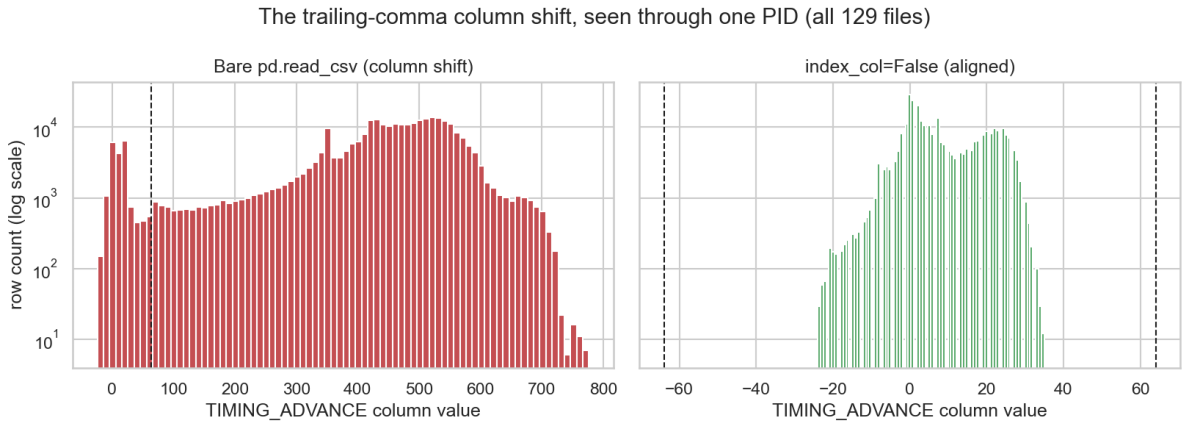


Figure 2: The column-shift defect seen through one PID. Left: under a bare `read_csv`, the TIMING_ADVANCE column contains a large population far outside the ±64° OBD-II bound (dashed lines), peaking at 776.6° — catalyst temperatures wearing the wrong column name. Right: with `index_col=False` the same files yield −24° to +35°, entirely inside the physical range. Note the logarithmic count axis.

The repair has three parts, and all three stay in the codebase. First, `index_col=False` tells the parser never to promote a column, which aligns all 27 columns in every file. Second, a numeric coercion pass converts stray non-numeric cells (one file hides a lone space character in the intake-air-temperature column) to missing values instead of letting one bad cell change a whole column's type. Third — the part we consider non-negotiable — a physical-bounds guard re-checks the four signature PIDs on every load and raises a loud error on violation, so any future misalignment fails at the loader instead of entering training as scrambled "healthy" data. A regression test locks the guard in place. The fix took the usable corpus from 9 files (∼4.4 h) to 129 files (∼84.5 h), a 19× expansion, and Figure 2 is the before/after evidence.

Frankly, this bug taught us more about data engineering than any lecture could: the failure mode was silent, the symptom appeared two layers away from the cause, and the wrong-but-plausible explanation (bad firmware) survived for weeks because it fit the evidence we had.

3.3.2 Working PID set

Four of the 27 channels are dead in this corpus and are dropped at load time: the commanded equivalence ratio is always zero (the Etios ECU does not report it), the two MIL counters are always zero (healthy car, lamp never lit), and warm-ups-since-codes-cleared is pinned at the OBD sentinel 255. Of the remaining channels, 14 form the working feature set (`USEFUL_PIDS`): RPM, speed, throttle, engine load, coolant temperature, both fuel trims, MAP, both accelerator-pedal channels, commanded throttle, intake air temperature, timing advance, and control module voltage.

3.4 Fault Injection Engine

The injection engine is the project's most technically serious contribution, and it exists for a blunt reason: the faults we want to detect are not for sale in any public dataset, so we had to write them — without writing physics fiction. The governing rule, enforced in code, is that the engine never emits a sensor value that violates engine causality. Fuel trims are clamped to the OBD-II ±25 % range, coolant may not change faster than 1°C per second (thermal inertia), MAP may not exceed barometric pressure on a naturally aspirated engine, throttle stays within 0–100 %, and timing stays within its normal −10° to +40° envelope. The full source is Appendix A Listing 1.

Each of the four fault models encodes a specific mechanical argument.

Air system (vacuum leak). Our first implementation followed the textbook: a leak means unmetered air, so short-term fuel trim swings strongly positive. We rejected that model after checking it against the engine we actually train on. The Etios reports MAP, load, and barometric pressure but no MAF — it is a *speed-density* engine, which computes airflow *from* manifold pressure. On such an engine the MAP sensor measures the post-leak pressure directly, the ECU recomputes airflow from it, and fuel largely self-compensates; service literature for MAP-only engines describes a slightly raised idle as often the only symptom. The corrected model therefore raises idle RPM (the leak behaves like a cracked-open throttle blade, about 15 rpm per unit of leak magnitude, clamped to a 1,500 rpm idle ceiling), raises idle MAP slightly, raises calculated load slightly, and applies only a small, idle-gated fuel-trim bump that washes out entirely off idle. Every effect is weighted by an idle factor that goes to zero as load rises, because a leak that is 20 % of airflow at idle is noise at wide-open throttle.

Fuel system (injector clog / low rail pressure). Less fuel per spray cycle produces a chronic lean condition that the ECU corrects. The detail that matters — and that a naive injector gets wrong — is the division of labor between the two trims. Short-term trim is the fast corrector: it *leads* during the developing fault. Long-term trim is the learned correction: it integrates upward and *holds*. Once the fault stops developing, STFT bleeds back toward zero (still oscillating) over roughly a 60-second constant while LTFT retains the full offset. Our `_steady_state_trim` helper implements exactly this handoff, so a developed fuel fault presents as high LTFT with near-zero mean STFT — the relationship a real adaptive ECU produces. MAP is deliberately left untouched: an injector restricts fuel, not air, and that untouched MAP is what separates a fuel fault from an air fault in feature space. A small idle-only RPM jitter models the roughness of an unevenly fed engine.

Coolant temperature sensor (stuck cold). The reported temperature drifts toward a stuck value (default 42°C), with the 1°C-per-second clamp applied sample by sam-

ple so the trajectory respects thermal inertia even in step mode. The ECU consequences follow: timing is retarded in proportion to the perceived cold deficit (up to 5°), and — the consequence our first version missed entirely — the ECU holds cold-start enrichment because it believes the engine never warmed up, so once closed-loop operation is reached the fuel trims go *negative* (rich), delivered through the same STFT-to-LTFT handoff. A stuck-cold sensor without the rich bias would be a signature no real car produces.

Throttle position sensor (potentiometer wear). The reported throttle angle drifts multiplicatively (default full-ramp factor 1.35), but only on rows where the accelerator pedal exceeds 10 % — a worn potentiometer track is not loaded at closed throttle, so idle rows stay clean. The commanded-throttle channel is deliberately left unchanged, which makes reported-versus-commanded divergence a second, independent signal on top of the throttle-to-pedal ratio.

Two properties hold across all four models. Degradation is a *ramp*: severity rises linearly from zero at a configurable onset (default 40 % into the session) over a configurable width (default 15 %), then holds — modeling gradual wear, with step mode as the one-row limiting case. And every injection is reproducible: the random seed, onset index, ramp length, magnitude, and noise level are stored in the output's metadata, so any training window can be traced to exactly the parameters that produced it.

One threat looms over all of this, and we measured it instead of waving at it. The classifier learns to invert the injector; the injector defines the answer key. If the model merely memorizes our coefficients, its scores are meaningless for any fault we did not author. The withheld-configuration experiment in Section 4.1 trains on one full parameter configuration and tests on a shifted one — different magnitudes, onset, ramp width, and noise — and the observed macro-F1 gap is 0.033. Small, but not zero, and we report every synthetic score in this book with that caveat attached: they are a self-consistency floor, not field detection.

3.5 Feature Engineering

The 60-second window and 10-second stride are not arbitrary. Sixty seconds is long enough to capture the slow signatures the faults live in — a fuel trim settling to a new level, a coolant curve failing to rise — while staying short enough that a driver's state (idling, accelerating, cruising) is roughly constant across the window, which keeps the regime features meaningful. The 10-second stride means consecutive windows overlap by 50 seconds, giving a sixfold data-augmentation effect over non-overlapping windows. That overlap is also precisely why the evaluation of Section 4.1 must split by session: two windows ten seconds apart are almost the same window, and letting one train while the other tests is self-examination, not generalization.

The classifier never sees raw rows. Each 60-second window (stride 10 s) is summarized into 83 features in four groups:

- **Per-PID statistics (70).** For each of the 14 working PIDs: mean, standard deviation, minimum, maximum, and first-to-last delta.
- **Cross-PID ratios (4).** `THROTTLE_TO_PEDAL_RATIO` (the TPS fault's home signal, computed as the median per-row ratio over pedal-active rows only, because the mean ratio explodes at idle); `MAP_PER_THROTTLE` (airflow sanity, open-throttle rows only); `FUEL_TRIM_DIVERGENCE` (LTFT − STFT, which directly encodes the handoff state); and `THROTTLE_CMD_ACTUAL_DELTA` (reported minus commanded throttle).
- **Trajectory features (4).** `COOLANT_WARMUP_RATE` in °C/min (a stuck sensor warms at zero rate); `FUEL_LOOP_ACTIVE` (closed-loop indicator); `RPM_IDLE_DRIFT` (idle stability); `TIMING_VS_TEMP` (timing deviation from the temperature-implied norm).
- **Operating-regime one-hots (5).** cold_start / warmup / idle / accel / cruise, detected from coolant, speed, and pedal variance, so the model can condition every other feature on context.

The representation's load-bearing property is what happens *after* extraction: every feature is z-scored against the source vehicle's own healthy baseline (mean and standard deviation fitted on designated healthy data). The model never reasons about "LTFT = 15 %"; it reasons about "LTFT is 4σ above normal *for this car*." Raw values differ between a 2014 Etios and a 2015 Yaris for perfectly healthy reasons — deviations from each car's own normal are comparable. This single decision is the cross-vehicle transfer mechanism, and it reduces adapting the system to a new vehicle to fitting one scaler on a few minutes of healthy driving.

3.6 Model Development

XGBoost classifier (Pillar 1). The production classifier is an XGBoost [9] multi-class model over the 83 normalized features: six classes (healthy, cold_start, air_system, fuel_system, coolant_temp_sensor, throttle_position_sensor), 300 trees of depth 6, learning rate 0.1, 80 % row and column subsampling, and the `multi:softprob` objective so downstream consumers get full per-class probabilities. Two weighting mechanisms shape training: samples are weighted inversely to class frequency (cold-start windows are scarce at about 5 %), and fuel-system samples carry an additional 0.5 down-weight — a correction added when that class over-fired (precision 0.457) in an early build. Training source: Appendix A Listing 2.

Random Forest baseline — kept as a warning label. Before XGBoost we trained a Random Forest [10] on an earlier five-class version of the task, and it scored a macro-F1 of exactly 1.000 on the held-out sessions. We kept that number in the repository deliberately. A perfect score does not mean the problem is solved; it means the synthetic task, as then configured, was trivially memorizable. The subsequent drop to 0.962 when we hardened the physics (the speed-density rewrite, the trim handoff, magnitude jitter) is evidence of rigor, not regression — the task got harder because it got more honest.

PID forecaster (Pillar 2a). Four XGBoost regressors, one per fault class's key channel, each predicting the *change* in that channel (in σ units) 60 seconds ahead: coolant temperature, MAP, the throttle-to-pedal ratio, and LTFT. Trained on healthy windows only. The residual between forecast and arrival is a model-agnostic drift signal: a sensor moving in a way the healthy dynamics cannot explain is suspicious regardless of which fault class it resembles. Each regressor is judged against the persistence baseline ("predict no change"), and one of the four fails that test: LTFT forecasting is 2.3× *worse* than persistence (Section 4.5). We document that failure rather than hide it, because LTFT is the single most diagnostic fuel-fault channel, and knowing the forecaster cannot anticipate it is operationally relevant.

IsolationForest anomaly detector (Pillar 2b). A one-class IsolationForest [7] trained on healthy windows over a 12-feature subset, emitting a 0–1 out-of-distribution score. Its test AUC of 0.69 is weak, and we say so; it is included as a complement to the classifier because it is the only component with any chance of reacting to a fault class we did not model.

The SHAP panel deserves a concrete example, because explanation is a real requirement in a diagnostic tool and not a decoration. When the classifier flags a fuel-system fault, the panel does not just show the label; it shows that (say) the long-term fuel trim mean contributed +0.43 toward that class, the fuel-trim divergence feature contributed +0.21, and the unchanged MAP contributed a smaller push away from air-system. A mechanic reading that does not need to trust the model on faith — the explanation names the two sensors a fuel fault should move and confirms the one it should not, and those are checkable claims. When the same panel instead attributes a prediction mostly to a regime one-hot or a voltage feature, that is a signal to distrust the call. Making the model's reasoning legible is what lets a human catch the model's mistakes, and it is why we pay the cost of exact TreeExplainer computation on every scored window rather than shipping a bare label.

StableAlerter (temporal vote). Raw per-window labels flicker, and a dashboard that flickers trains its user to ignore it. A fault alert therefore requires a majority across a three-window vote with hysteresis before it is raised or cleared. Two hard rules ride on top: non-fault labels (healthy, cold_start, warming-up) can never raise a fault alert, and

any label declared Untested for this vehicle is rewritten to healthy *before* it enters the vote — an untrustworthy prediction should not accumulate votes toward an alarm.

3.7 Dashboard

The operator-facing layer is a Streamlit [15] application that replays recorded CSVs at 1–50× speed or attaches to a live ELM327 stream. Top to bottom it renders: a full-width status banner (state, confidence, session time); a fault severity strip showing per-fault physics-derived severity bars — severity comes from closed-form formulas over the features, not from the forecaster, after the forecaster produced phantom 17–27 % severities on healthy cars; live sparklines for four key PIDs; a SHAP top-contributors panel translating feature attributions into workshop vocabulary; and a time-stamped alert log fed by the StableAlerter and the deterministic cold-start rule checker.

When a replayed file ends, the dashboard renders its primary output: the end-of-read health report. The report treats the first 20 % of the recording as the vehicle's own baseline and excludes it from the verdict (scoring the baseline against itself is circular). Over the remaining windows it computes, per fault, the share of windows won and the 95th percentile of physics severity. A fault is DETECTED only if it wins at least 40 % of evaluable windows *and* carries material severity; a fault that dominates labels at approximately zero severity is reported INCONCLUSIVE, because that combination is the signature of a feature artifact, not physics. Fewer than 10 evaluable windows yields INSUFFICIENT DATA rather than a verdict. Every report carries its caveats in the rendered output, including the one that matters most: this design detects a fault that *develops during the recording*; a fault already present at key-on becomes the baseline and reads as normal.

The report's Untested machinery deserves its own paragraph, because a real failure created it. Different cars expose different PIDs, and each fault class declares the PIDs its primary signature requires (Appendix A Listing 4): air_system requires MAP; TPS requires throttle, commanded throttle, *and both* pedal channels; and so on. When we first scored our healthy MAF-based Yaris — a car with no MAP sensor — the pipeline filled the missing MAP features with the training vehicle's baseline values, the classifier read that fabricated flatness as a massive deviation, and the dashboard announced a 97 % air-system fault on a healthy car. The false positive, not theory, drove the redesign: a missing PID now yields NaN features (which XGBoost routes through learned default splits), the affected fault class is marked Untested for the whole session, its label is suppressed from alerting, and the end-of-read report prints "UNTESTED — PID unavailable" where a score would have been. Fabricated sensor values produce confident wrong answers; refusing to score is the honest alternative, and Chapter 4 shows it working on the same car that broke the first version.

4 Experimental Results and Evaluation

4.1 Evaluation Methodology

Adjacent 60-second windows cut at a 10-second stride share 50 of their 60 rows. Split them randomly and the same seconds of driving land on both sides of the train/test boundary; the model then recognizes drives rather than faults, and macro-F1 inflates by 10–20 points in our measurements. Every split in this chapter is therefore made at the level of entire recording sessions — a session's windows go to one side, whole — and a regression test in the repository fails the build if any session ever appears on both sides. This is the one evaluation rule we treat as beyond negotiation.

We report three evaluation paths and rank them by how much trust each deserves.

1. **Fixed holdout (lowest trust).** Two sessions (drive1, live12) held out, the rest trained on. Fast and reproducible, but the 39 `live` commute sessions are near-duplicates of one another — the same route, driver, and car — so a held-out `live` session has close siblings in training. Numbers from this path are a self-consistency ceiling and we do not headline them.

2. **Leave-one-session-out (LOSO) cross-validation.** All 129 sessions held out in turn. Honest within-vehicle, still synthetic.

3. **Leave-one-family-out (LOFO) cross-validation.** All sessions of an entire family (e.g. every idle recording) held out together. This is the strongest context-transfer test the corpus supports: the model must classify faults in a driving context it has never seen.

– **Real-fault evaluation (the missing path).** The only path that would demonstrate actual fault detection. Its evaluation code exists in the repository; its data folders are empty, because no public real-fault OBD-II recordings exist at 1 Hz and we did not induce faults on our own vehicles. We list it to be explicit about what has *not* been shown.

One additional experiment probes the self-consistency problem directly. The withheld-configuration test trains the classifier on faults injected with one full parameter set (magnitudes, onset 40 %, ramp 15 %, noise 0.3) and evaluates on faults injected with a deliberately different one (shifted magnitudes, onset 55 %, ramp 25 %, noise 0.5). Same-config macro-F1 is 0.892; withheld-config is 0.859; the gap is 0.033. The model is not simply memorizing one coefficient table, but the non-zero gap is a measured reminder that some generator overfit exists.

4.2 Classification Results

Table 4 collects the headline numbers across evaluation paths, worst cases included.

Table 4: Classifier results across evaluation paths. All numbers measure recovery of synthetically injected faults (self-consistency), not real-fault detection.

Evaluation	Macro-F1	Reading
Random Forest baseline (fixed holdout)	1.000	Memorization signal, not performance
XGBoost (fixed holdout)	0.962	Near-duplicate session leak; not headlined
XGBoost LOSO mean (129 folds)	0.917	Spread $\sigma = 0.084$
XGBoost LOSO worst fold (live31)	0.699	The honest synthetic floor
XGBoost LOFO mean (5 families)	0.977	Context-transfer consistency
XGBoost LOFO worst family (live)	0.958	**Headline number**

Two rows in that table deserve slow reading. The Random Forest's perfect 1.000 on the earlier five-class task is the reason this book never celebrates a high synthetic score: when your own generator writes the answer key, perfection is a symptom. The XGBoost drop to 0.962 arrived *after* we corrected the injector's physics and added per-session magnitude jitter — the score went down because the task stopped being easy to memorize, which is the direction of progress here.

We headline the LOFO worst family (0.958) and name it carefully: *context-transfer consistency*. Held out on an entire driving context — all 39 commute sessions, 30,336 windows — the classifier still recovers the injected faults it was trained to recognize in other contexts. That is a real property worth having. It is not fault detection, because in every fold, train and test faults alike were authored by the same injector. Table 5 gives the per-class breakdown for that worst family; the full per-family table is in Appendix C.

Table 5: Per-class F1 for the worst LOFO fold (live family held out; 39 sessions, 30,336 test windows).

Class	F1
healthy	0.933
air_system	0.965
fuel_system	0.917
coolant_temp_sensor	1.000
throttle_position_sensor	0.932
cold_start	1.000

The LOSO distribution tells the more cautious story: a mean of 0.917 hides a standard deviation of 0.084 and a worst fold of 0.699 (session live31). Short sessions with unusual driving produce genuinely hard folds, and any deployment estimate should budget for the floor, not the mean.

The per-class numbers repay a closer look, because they reveal that "the model" is really six detectors of very different difficulty stapled together. Coolant-sensor detection is effectively solved on synthetic data: F1 sits at 1.000 in five of the six LOFO folds and never drops below 0.999. The reason is physical and worth stating plainly — a stuck-cold sensor produces a coolant trajectory that simply does not occur in healthy driving. A real engine warms monotonically toward 88–90°C and stays there; a stuck sensor holds a flat, too-low reading while everything else about the drive looks normal. At 1 Hz that contradiction is unmistakable, and no amount of driving-context variation blurs it, which is exactly why the class survives family holdout untouched.

Fuel-system detection is the mirror image, and its 0.917 F1 in the worst family is the lowest fault-class number in Table 10 for a reason. A developing fuel fault expresses through the same channel a mild air leak does — the fuel trims — and in the early ramp, before LTFT has fully integrated, the two look alike. The 0.825 precision on fuel in the fixed-holdout breakdown (Table 8) confirms it: the model casts a wide net for fuel and catches some air and some healthy windows in it. We accept this as a structural floor rather than a tuning failure. The fuel down-weight of Section 3.6 pulls the boundary back, but it cannot separate two faults that genuinely share a signature during their first thirty seconds. The honest reading is that our system distinguishes *fuel-or-air trim disturbance* from healthy very well, and resolves *which* of the two only once the fault matures.

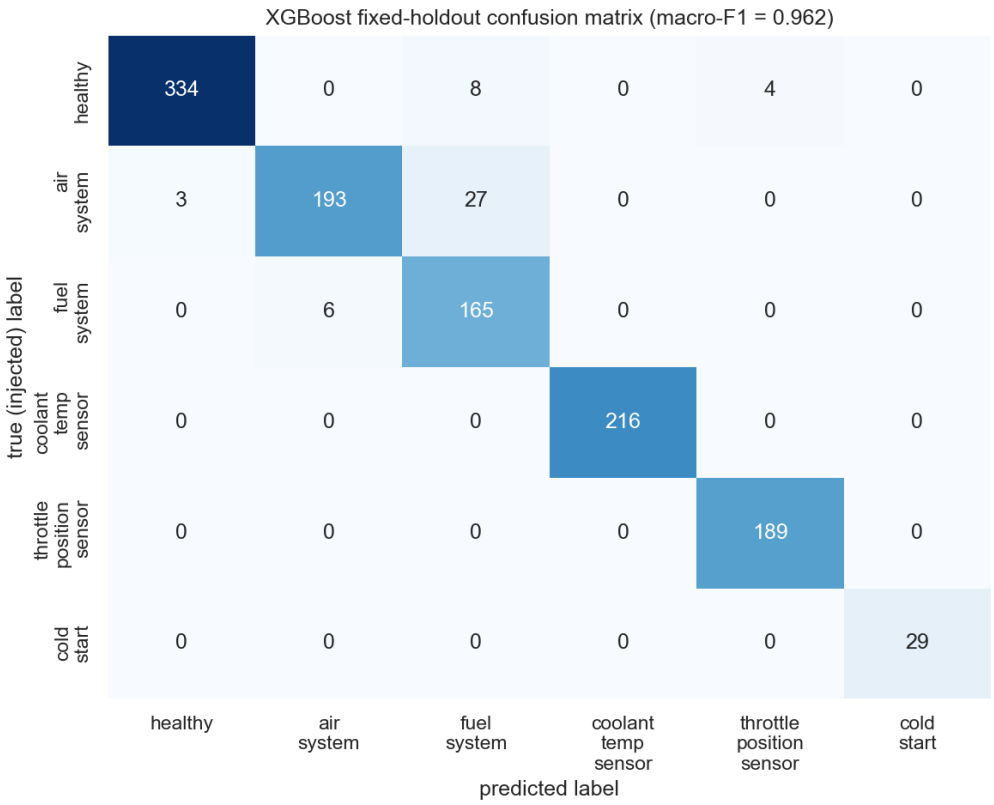
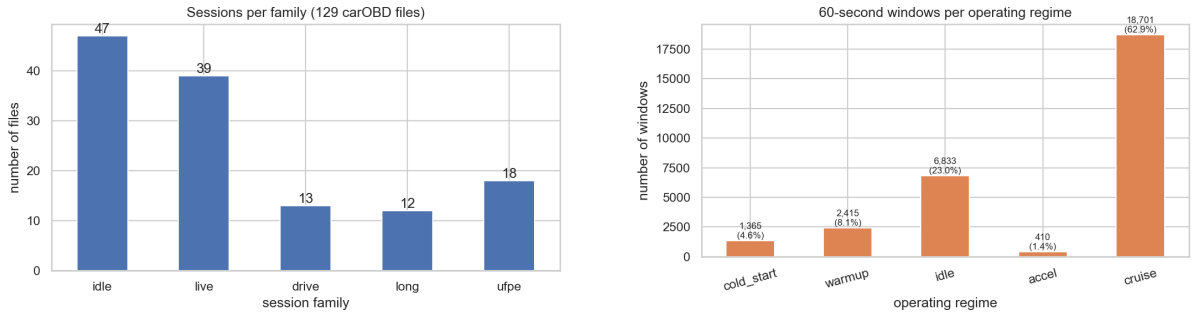


Figure 3: Fixed-holdout confusion matrix for the six-class XGBoost classifier (sessions drive1 + live12; macro-F1 = 0.962).

Figure 3 shows where the errors live: almost entirely inside a healthy–air–fuel triangle. The largest single cell is air_system predicted as fuel_system (27 of 223 air windows), with smaller flows of healthy into fuel (8) and fuel into air (6). The triangle is physically sensible. Early-ramp air and fuel faults both surface first as small fuel-trim disturbances at idle, and a window straddling the fault onset is legitimately ambiguous between "still healthy" and "barely faulty." Coolant-sensor and cold-start windows, whose thermal signatures are unambiguous at 1 Hz, are never confused with anything (F1 = 1.000 and support 216 and 29 respectively).

4.3 Data Exploration Results

The exploratory analysis below was produced by the repository's Phase-1 notebook against the recovered 129-file corpus; the figure-generation script in `docs/book/figures/` reproduces every plot.



(a) Sessions per family.

(b) Windows per operating regime.

Figure 4: Corpus composition on both axes. By file count the idle family dominates (47 of 129); by 60-second windows, cruise dominates (62.9 %) because driving sessions are far longer than idle ones.

Composition is imbalanced on both axes, and the two axes disagree (Figure 4). Idle recordings are the most numerous *files*, but cruise dominates the *window* count at 62.9 %, followed by idle at 23.0 %, warm-up at 8.1 %, cold-start at 4.6 %, and acceleration at just 1.4 %. The scarce regimes matter disproportionately: a TPS fault only expresses under pedal load, and cold-start windows are the entire training basis for that class. Class balancing at training time addresses the label imbalance; the accel-regime scarcity is a property of 60-second windows over real driving that balancing cannot fix.

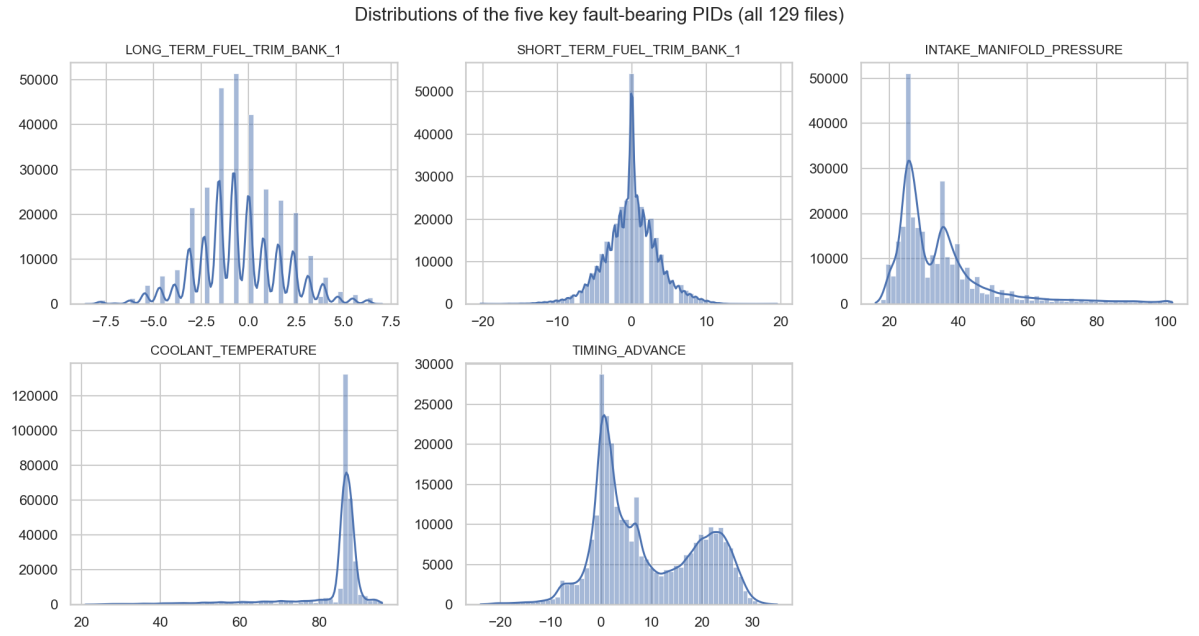


Figure 5: Distributions of the five key fault-bearing PIDs across all 129 files. Fuel trims are roughly symmetric and slightly rich-shifted with visible OBD-II quantization; coolant is bimodal (cold-start shoulder plus a warm-operation peak near 88–90°C); MAP is multimodal from the idle/driving mixture; timing sits entirely inside its physical envelope.

The distribution shapes (Figure 5) argue for the pipeline's normalization choice: mul-

timodal, skewed channels do not reward any Gaussian assumption on raw values, whereas per-vehicle z-scoring measures displacement from the car's own mixture. The coolant bimodality confirms cold-start data is well represented, and the timing histogram — bounded in $[-24°, +35°]$ across the entire corpus — is the standing proof that the column-shift repair holds (Figure 2 showed the same channel reaching 776.6° under the defective parse).

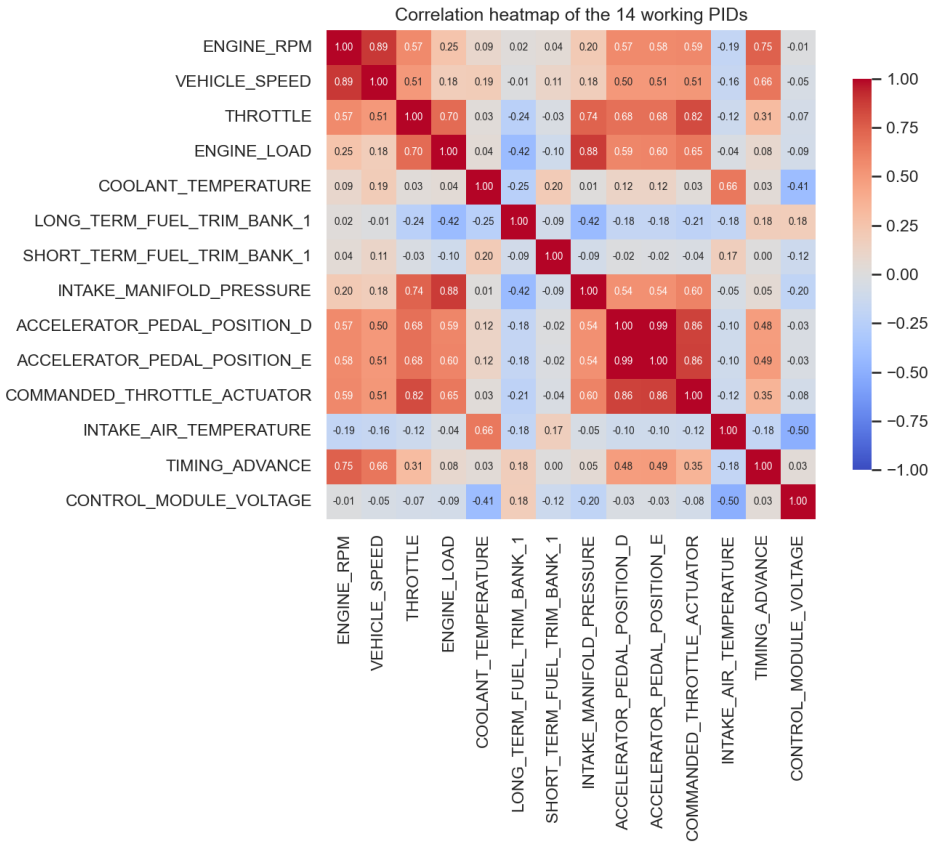


Figure 6: Correlation heatmap of the 14 working PIDs over the full corpus.

The correlation structure (Figure 6) shows the expected physical couplings — RPM with speed, MAP with engine load, and a tight throttle/pedal cluster in which the two accelerator-pedal channels are near-duplicates of each other. We keep both pedal channels despite the redundancy: their *agreement* is exactly what a TPS fault disturbs, and the availability contract of Section 3.7 treats them as jointly required for that class.

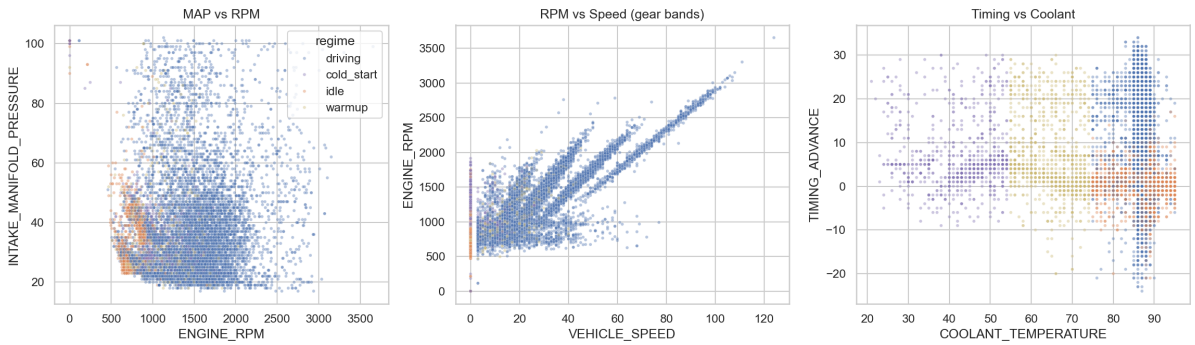


Figure 7: Bivariate structure, colored by a row-level regime proxy. Left: MAP vs RPM separates the idle cluster from the driving cloud — the region the air-system fault perturbs. Center: RPM vs speed resolves into discrete gear bands, a drivetrain sanity check that a residual column shift would scramble. Right: timing advance climbs and stabilizes as coolant warms — the coupling the coolant-sensor fault exploits.

The bivariate views (Figure 7) close the loop between data and fault design: three of the four injected signatures are visible as structure in the healthy data itself. Clean gear rays in the center panel across all 129 files are also the most legible visual proof that the recovered corpus is physically coherent. Duplicate rows, at 29,329 (9.6 %), are dominated by legitimate steady-state repeats at 1 Hz — an idling engine reports the same values for many consecutive seconds — so we do not drop them; deleting idle repeats would distort exactly the statistics the idle-gated air-system model depends on.

4.4 Real-Vehicle Testing

The deployment target was a healthy 2015 Toyota Yaris — a different car, a different engine-management philosophy (MAF-based), and a different decade from the training vehicle. Data reached the pipeline through the OBD-LOG app and an ELM327 Bluetooth adapter. We scored two recordings: `obd22`, a 1,740-row (29-minute at 1 Hz) session used as the baseline capture, and `transition`, a 2,148-row session spanning a cold start and the transition into driving. The app's three tier captures (124, 121, and 65 rows at parked idle) served as the throughput tests of Table 3.

The single missing channel dominates everything about this test. On both recordings, exactly one working PID is absent: `INTAKE_MANIFOLD_PRESSURE`. The Yaris meters air with a MAF sensor and simply does not expose PID 010B, so the MAP column is present in the CSV schema but empty in every row.

Table 6: Real-vehicle scoring outcomes on the healthy 2015 Toyota Yaris, with the Untested contract applied. Raw label counts are the classifier's per-window outputs before suppression.

Session	Rows / windows	Missing PIDs	Evaluable windows	Verdict
obd22	1,740 / 169	MAP only	7 of 169	INSUFFICIENT DATA
transition	2,148 / 209	MAP only	6 of 209	INSUFFICIENT DATA

Behind Table 6 sits the incident that reshaped the system. Before the Untested contract existed, missing MAP features were filled with the *training vehicle's* baseline values. To the classifier, a perfectly flat, perfectly average MAP on a car whose every other channel varied normally looked like an enormous deviation — and the model mapped that artifact squarely onto air_system. The healthy Yaris scored "97 % air_system, FAULT DETECTED." The raw label counts still show the shape of that failure: on `obd22`, 162 of 169 windows carry the air_system label (with 5 healthy, 1 fuel, 1 TPS); on `transition`, 203 of 209. Fabricated sensor values produce confident wrong answers — that is the design lesson we consider this test's main deliverable.

With the contract in place, the same recordings produce the correct behavior end to end: MAP-dependent features stay NaN, air_system is declared Untested for the session, its windows are excluded from evaluation, and the seven (respectively six) remaining evaluable windows fall below the ten-window floor — so the report prints INSUFFICIENT EVALUABLE DATA instead of a verdict. We want to be precise about what this outcome is: it is not a failed test. The system was shown a car it cannot fully evaluate and said so, where its predecessor invented a fault. On a MAF-based car, honest air-system coverage requires a MAF-based feature path, and that is future work, not a rounding error.

The channels that *were* available behaved healthily. LTFT sat at -6.4 % mean (spread -8.3 to -4.5 %) across the Yaris recordings — a mild, steady rich bias, consistent across sessions, comfortably inside the ± 10 % band that our fuel-fault severity model treats as the concern threshold, and exactly the kind of per-vehicle personality the baseline z-scoring exists to absorb.

4.5 Forecaster Results

Table 7 evaluates the four PID-forecaster regressors against the persistence baseline ("the next 60 seconds change nothing"), in z-scored units on held-out sessions.

Table 7: PID forecaster mean absolute error (z-units, 60 s horizon, held-out sessions) versus the persistence baseline.

Target channel	Forecaster MAE	Persistence MAE	Beats baseline?
COOLANT_TEMPERATURE	0.050	0.112	Yes (2.3× better)
INTAKE_MANIFOLD_PRESSURE	0.773	0.800	Barely
THROTTLE_TO_PEDAL_RATIO	0.829	0.903	Barely
LONG_TERM_FUEL_TRIM	1.152	0.492	**No — 2.3× worse**

The pattern is physical. Coolant temperature follows slow, deterministic thermal dynamics, and the forecaster genuinely learns them. MAP and the throttle-to-pedal ratio are driver-driven; sixty seconds ahead, the driver is close to unpredictable, and the forecaster only marginally beats assuming nothing happens. LTFT is the painful row: its 60-second change on healthy data is near-zero noise around a session-specific level, the model overreacts to that noise, and it loses to persistence by a factor of 2.3 — on the one channel most diagnostic of fuel faults. We keep the LTFT regressor in the artifact set and its failure in this table, because a reader who deploys Pillar 2 must know that the drift signal for fuel faults comes from the anomaly detector and the classifier, not from the forecaster. For completeness, the one-class IsolationForest scores an AUC of 0.69 (95 % CI 0.66–0.72) separating healthy from any-fault windows on the synthetic test set, and its false-positive rate on held-out healthy sessions (15.6 % at the intended 1 % budget) is the measured argument for per-vehicle re-fit before any live use.

4.6 Discussion

Three threads run through these results. First, the evaluation hierarchy did its job: the same model that scores 1.000 (RF) or 0.962 (fixed holdout) under leaky-adjacent conditions scores 0.699 on its worst honest fold, and designing the report around the floor rather than the ceiling changed what we were willing to claim. Second, the cross-vehicle test produced no detection result at all — and that non-result, delivered as two correct refusals on a car the first version had confidently mislabeled, is the strongest evidence in this book that the reporting layer works. Third, every number above is bounded by the same asterisk: the faults were written by our own injector. The 0.033 withheld-configuration gap says the model is not merely memorizing coefficients; nothing in this chapter says it detects real faults. Closing that gap needs data, not modeling, and Section 5.3 puts it first in line accordingly.

Read together, the classification results, the forecaster failure on LTFT, and the Yaris deployment tell a consistent story rather than three separate ones. Wherever a signal

is slow and thermally or mechanically determined — coolant warm-up, timing versus temperature — both the classifier and the forecaster do well, because the underlying physics leaves little room for session-specific noise to hide in. Wherever a signal reflects the ECU's learned, car-specific fuel state — LTFT above all — both components struggle, because the same absolute trim value means something different on every car and every tank of fuel. That split is not a coincidence of which experiments happened to work; it is the same physical fact showing up three separate times, and it is the strongest argument in this report for treating per-vehicle baselining as necessary rather than optional.

5 Conclusion and Future Work

5.1 Conclusions

We set out to catch a developing engine fault from the standard 1 Hz OBD-II stream, before a diagnostic trouble code fires. What we built is a physics-grounded fault injection engine for four common fault classes; a vehicle-agnostic feature and normalization pipeline that lets a model trained on one car run on another; a six-class XGBoost classifier evaluated under session-level and family-level cross-validation; a per-PID forecaster and a one-class anomaly detector as a second, independent line of evidence; a native Android application, OBD-LOG, for real-vehicle data collection over Bluetooth; a live Streamlit dashboard that explains its own predictions and closes every session with a plain-language health verdict; and, threaded through all of it, a reporting layer that says "Untested" or "insufficient data" instead of manufacturing a number it cannot back.

What we did not build is a validated real-fault detector. That distinction matters more than any F1 score in this book, and we have tried not to blur it anywhere above. Every classification number in Chapter 4 measures the model's ability to recover labels written by our own injector. Building the validated version needs one thing our project could not supply on its own timeline: labeled recordings of real, induced faults. Section 5.3 makes that the first line of future work, not the last, because it is the single change that would do the most.

If this project has one engineering contribution worth naming beyond the models themselves, it is the discipline of refusal. Twice in this book the system was handed a situation it could not honestly resolve — a vehicle missing the sensor a fault class depends on, a recording too short to support a verdict — and twice it said so instead of guessing. The alternative version of this system, the one we shipped first and then un-shipped, guessed with 97 % confidence on a healthy car. We consider the version that admits what it does not know to be the more useful tool, even though it produces fewer confident-looking numbers for a slide.

That discipline did not come free. It cost us a working demo we had to throw away, a redesign of the reporting layer around an evaluability contract nobody had asked for at project kickoff, and an uncomfortable week of explaining to each other why a 97 % confidence score was the project's worst result rather than its best. We record that cost here because a reader building a similar system will hit the same choice — ship the number the model gives you, or build the machinery to know when that number is not trustworthy — and the second path is more work for a less impressive-looking demo. We think it is the only defensible path for anything a mechanic or a driver might actually act on.

The broader lesson carries past this particular car and these particular four faults.

Any system trained on synthetic labels inherits the honesty of its label generator, and any system deployed across more than one instance of its target domain inherits the assumptions baked into its first training example. We tried to make both of those inheritances visible rather than implicit: the injector's physics reasoning lives next to its code, the withheld-configuration gap is a number in this book and not a footnote, and the cross-vehicle failure that produced the Untested contract is documented as a failure, not smoothed into a success story after the fact. If a future team extends this work, we would rather they inherit the visible assumptions than have to rediscover them the way we did.

5.2 Limitations

We list these without softening them, because the audience for this report already knows the honest framing and overselling would cost more credibility than it could ever buy back.

1. **Every fault metric in this book measures self-consistency, not field performance.** The injector writes the fault; the classifier learns to invert the injector. A macro-F1 of 0.958 on a held-out driving family says the model generalizes across driving contexts for faults it was told to recognize. It does not say the model would catch a real vacuum leak on a real car. No experiment in this project contradicts that limitation, and none was designed to.

2. **The model has never detected a real fault.** Not once, on any vehicle, in any recording. The real-fault evaluation code exists in the repository and its data folder is empty. This is the single most important gap between what we built and what a mechanic would need to trust it.

3. **Air-system detection is unavailable on the majority of modern cars.** MAF-based engines — our own test vehicle among them — expose no MAP sensor, and the air-system fault's primary signature lives on MAP. On those cars the class is correctly marked Untested, but correctly refusing to answer is not the same as answering.

4. **Phone-based sampling aliases fast dynamics.** Our measured ELM327 poll rate falls to 0.37 Hz at 21 polled PIDs — well under the 1 Hz the training pipeline assumes. Results scored on app-captured data should be read as pessimistic relative to a faster logger, because slow polling smears short fuel-trim oscillations into apparent stability, which can understate real trim variance and, in the other direction, overstate false-positive risk on transient events the sampler simply missed.

5. **Self-baselining cannot see a fault that predates the recording.** The end-of-read report treats the first 20 % of a session as healthy ground truth. A car that is already faulty at key-on has that fault baked into its own baseline and will read as normal for the rest of the drive. This is a structural property of the self-baseline design, not a bug to be patched.

5.3 Future Work

The items below are ordered by impact, not by how soon they could be started. Each one names a specific mechanism and a specific reason it was not already done, because "improve the model further" is not a plan.

1. **Real fault data.** This is the one change that would convert the project from "a detection algorithm plus a collection protocol" into a validated detector. A single matched set of before, during, and after recordings of a genuinely induced fault — a vacuum line partially disconnected under a mechanic's supervision, say — would let the real-fault evaluation code that already exists in the repository produce its first non-empty result. We did not do this in the current phase because it requires a donor vehicle, a supervising mechanic, and a written abort protocol we were not yet in a position to execute safely; the collection protocol documenting exactly how to do it is already written and waiting in the repository, alongside the scorer that would consume its output.

2. **A live Android-to-server pipeline.** Route the app's CSV output over HTTPS, in roughly 60-second chunks, to a cloud-hosted version of the existing FastAPI backend, which already implements JWT authentication. The server holds one persistent scoring session per vehicle — one fitted baseline, every chunk scored against it in sequence — and the Streamlit dashboard mirrors that session live while the phone shows a simplified health card. This was not built now because it needs a hosting decision (which cloud, which uptime guarantee) that is out of scope for a laptop-based thesis defense, and because it depends on item 1 producing enough real sessions to be worth streaming.

3. **Apache Kafka for multi-vehicle ingest.** At fleet scale, direct HTTP ingest does not survive a phone losing signal mid-upload, and a new server process per vehicle does not scale. The natural next architecture makes the Android app a Kafka producer, runs a consumer group that performs inference per vehicle, and writes a second consumer to a time-series store for the dashboard to query. Kafka's offset-based replay absorbs cellular dropouts that a plain HTTP POST cannot, and a Kafka Connect OBD-II source connector is the natural long-term target. This

is future work rather than current work because it only pays for itself once item 2 exists and more than one or two vehicles are actually streaming — building it first would be premature infrastructure for a single-laptop demo.

4. **Per-regime baselining.** Right now one healthy baseline covers idle and driving alike, and we measured the resulting idle-to-drive feature shift at a mild 2–3σ, concentrated on the throttle side, and already absorbed by the regime one-hot features and the alerter's temporal vote. Because the measured shift is mild, this is a robustness improvement queued behind the higher-impact items above, not a critical bug demanding an emergency fix. Implementing it properly means retraining with regime-conditioned normalization rather than patching the existing scaler.

5. **Intermittent and accelerating fault shapes in the injector.** Every fault the injector produces today follows one shape: a linear ramp to a held plateau. Real components do not always fail that way — a connector that only opens under vibration produces an intermittent signature, and bearing-style wear often follows a Weibull-shaped acceleration rather than a straight line. Widening the injector's shape library would reduce the risk that the classifier is quietly overfitting to "ramp-then-hold" as a pattern rather than learning the underlying fault physics. We scoped this out of the current phase because the four existing fault models, in ramp and step mode, already consumed the available time for physics validation, and adding shapes without re-validating physics for each one would reintroduce exactly the kind of unaudited synthetic data this project was built to avoid.

6. **A session-label field in the Android app.** A single text field, written as one more CSV column, so a recording can declare itself *baseline*, *fault-type-X*, or *healthy-post-repair* without an external notebook tracking which file is which. It is a small UI change, deliberately left until last because it only becomes necessary once real-fault collection (item 1) is actually underway — there is no annotation burden to solve before then.

References

[1] D. Rimpas, A. Papadakis, and M. Samarakou, "OBD-II sensor diagnostics for monitoring vehicle operation and consumption," *Energy Reports*, vol. 6, pp. 55–63, 2020.

[2] U. Shafi, A. Safi, A. R. Shahid, S. Ziauddin, and M. Q. Saleem, "Vehicle remote health monitoring and prognostic maintenance system," *Journal of Advanced Transportation*, vol. 2018, pp. 1–10, 2018.

[3] E. J. Maranhão, "carOBD: OBD-II data logging dataset from a Toyota Etios 2014," GitHub repository, 2019, https://github.com/eron93br/carOBD, accessed July 2026.

[4] A. Theissler, J. Pérez-Velázquez, M. Kettelgerdes, and G. Elger, "Predictive maintenance enabled by machine learning: Use cases and challenges in the automotive industry," *Reliability Engineering & System Safety*, vol. 215, p. 107864, 2021.

[5] R. Prytz, S. Nowaczyk, T. Rögnvaldsson, and S. Byttner, "Predicting the need for vehicle compressor repairs using maintenance records and logged vehicle data," *Engineering Applications of Artificial Intelligence*, vol. 41, pp. 139–150, 2015.

[6] A. Theissler, "Detecting known and unknown faults in automotive systems using ensemble-based anomaly detection," *Knowledge-Based Systems*, vol. 123, pp. 163–173, 2017.

[7] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proceedings of the Eighth IEEE International Conference on Data Mining (ICDM)*. IEEE, 2008, pp. 413–422.

[8] M. Vergara, L. Ramírez, and H. Combita-Niño, "EngineFaultDB: A novel dataset for automotive engine fault classification and baseline results," *IEEE Access*, vol. 11, pp. 126 155–126 165, 2023.

[9] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.

[10] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[11] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017, pp. 4765–4774.

[12] SAE International, "J1979: E/E diagnostic test modes," Society of Automotive Engineers, Surface Vehicle Standard, 2017.

[13] International Organization for Standardization, "ISO 15765-4: Road vehicles — diagnostic communication over controller area network (DoCAN) — part 4: Requirements for emissions-related systems," ISO, Tech. Rep., 2021.

[14] B. Whitfield, "python-OBD: OBD-II serial module for reading engine data," Software, version 0.7.2, 2019, https://github.com/brendan-w/python-OBD, accessed July 2026.

[15] Streamlit Inc., "Streamlit: A faster way to build and share data apps," Software, version 1.38.0, 2024, https://streamlit.io, accessed July 2026.

[16] S. Ramírez, "FastAPI: A modern, fast web framework for building APIs with Python," Software, version 0.115.6, 2024, https://fastapi.tiangolo.com, accessed July 2026.

A Key Source Code Listings — Analysis Pipeline

The listings below are included verbatim from the project repository (`Predictive-MaintenanceV1`) at the revision this book was built from. They are the four modules the main text leans on most: the fault injection engine (Section 3.4), the XGBoost training module (Section 3.6), the end-of-read session report (Section 3.7), and the PID availability contract that implements the Untested behavior.

```
"""Physics-constrained fault injection for OBD-II time-series data.

Ramp mode: each fault degrades linearly from zero to full magnitude over
`ramp_len` seconds, then holds at full magnitude. Models gradual wear or
progressive sensor degradation.

Step mode: ``ramp_len = 1``---fault is fully developed one row after onset.
Physics clamps (1 °C/s coolant inertia, ±25 % fuel trim, etc.) still apply.

Usage
-----
    from src.injection.fault_injector import inject_fault, inject_session,
        ↪ InjectionParams

    df_clean = load_carobd_csv("data/raw/carOBD/drive1.csv")
    df_faulty = inject_session(df_clean, "fuel_system", random_seed=42)
    print(df_faulty.attrs["fault_label"])   # "fuel_system"
    print(df_faulty.attrs["injection"])     # InjectionParams(...)
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Callable, Literal

import numpy as np
import pandas as pd

FaultType = Literal[
    "air_system",
    "fuel_system",
    "coolant_temp_sensor",
    "throttle_position_sensor",
]

# --- Physical bounds (OBD-II standard + engine physics) -----------------------
# Changing these values must be justified by a spec reference or audit finding.
_STFT_MAX = 25.0           # % (OBD-II standard range ±25 %)
_LTFT_MAX = 25.0           # %
_THROTTLE_MIN = 0.0        # %
_THROTTLE_MAX = 100.0      # %
_COOLANT_MIN = 35.0        # °C
_COOLANT_MAX = 130.0       # °C
_COOLANT_MAX_DELTA = 1.0   # °C per sample (thermal inertia---cannot spike instantly)
_TIMING_MIN = -10.0        # ° BTDC
_TIMING_MAX = 40.0         # ° BTDC
_BARO_FALLBACK = 101.0     # kPa---used when ABSOLUTE_BAROMETRIC_PRESSURE absent

```

```python
# Sensible defaults for each fault at "moderate" severity
_DEFAULT_MAGNITUDE: dict[str, float] = {
    "air_system": 13.0,               # kPa-equivalent leak size (drives idle RPM + MAP, see _inject_air_system)
    "fuel_system": 18.0,              # % LTFT bias at full ramp
    "coolant_temp_sensor": 42.0,      # °C stuck-sensor target temperature
    "throttle_position_sensor": 1.35, # multiplier on THROTTLE at full ramp---widened by _TPS_DEADBAND so full-ramp severity stays = 1.0
}

# -- Speed-density vacuum-leak constants (P0-1) --------------------------------
# carOBD reports MAP + ENGINE_LOAD + BARO and NO MAF PID → this is a
# speed-density (MAP-based) engine.  On such engines the MAP sensor measures
# post-leak manifold pressure directly, so the ECU re-computes airflow and
# largely self-compensates fuel.  The robust, reliable signatures of a vacuum
# leak are therefore mechanical, not fuel-trim:
#   1. raised idle RPM (the leak acts like a partly-open throttle blade),
#   2. slightly elevated MAP at idle / low load (less vacuum),
#   3. only a small, idle-only fuel-trim bump that washes out off-idle.
# Sources: Vehicle Service Pros "Fuel Trim for Diagnostics"; ScannerDanner
# ("MAP-only engines tend to be very tolerant of vacuum leaks ... often the
# only symptom is a slightly raised idle speed"); ASE Fuel-Injection Diagnosis.
_AIR_RPM_PER_KPA = 15.0       # idle RPM rise per unit leak magnitude at full ramp (13 → ~195 rpm)
_AIR_IDLE_RPM_CEILING = 1500.0  # a vacuum-leak idle-up rarely exceeds this; clamp for sanity
_AIR_IDLE_RPM_MAX = 1200.0    # rows above this RPM are not "idle" for the bump
_AIR_IDLE_SPEED_MAX = 2.0     # km/h---rows above this are not "idle"
_AIR_MAP_COEFF = 0.5          # MAP delta per leak magnitude (reduced from 1.0; idle-gated)
_AIR_STFT_COEFF = 0.15        # small idle-only STFT bump (cut hard from 0.8)
_AIR_LTFT_COEFF = 0.05        # LTFT marginal on speed-density (greatly shrunk from 0.32)
_AIR_LOAD_COEFF = 0.3         # calculated-load rise per leak magnitude (idle-gated)

# -- Stuck-cold ECT rich-bias constants (P1-2) ---------------------------------
# A stuck-cold ECT makes the ECU believe the engine never warmed up, so it
# holds cold-enrichment fuelling.  The rich mixture shows, once closed-loop is
# reached, as NEGATIVE fuel trims---the single most OBD-detectable consequence,
# and one the old injector (timing-retard only) never produced.  Without it a
# real stuck-ECT on the Skoda would present with a signature the model never saw.
_COOLANT_RICH_LTFT = -8.0       # % LTFT at full fault (chronic rich bias)
_COOLANT_RICH_STFT_PEAK = -4.0  # % STFT peak during the transient before handoff

# -- STFT→LTFT steady-state handoff (P0-3) -------------------------------------
# Real adaptive fuel control: STFT is the fast corrector that LEADS during a
# developing fault, then bleeds back toward ~0 (still oscillating) as LTFT
# integrates and HOLDS the persistent offset.  A developed fault therefore
# shows high LTFT and near-zero MEAN STFT.  Modelling both elevated together
# (the old behaviour) trains FUEL_TRIM_DIVERGENCE on a relationship the real
# ECU never reproduces.  Source: Foxwell "Understanding LTFT Bank 1".
_HANDOFF_TAU = 60.0           # rows (≈ seconds at 1 Hz) for STFT to decay post-ramp


def _steady_state_trim(
    ramp: np.ndarray,
    stft_peak: np.ndarray,
    ltft_full: np.ndarray,
    handoff_tau: float = _HANDOFF_TAU,
) -> tuple[np.ndarray, np.ndarray]:
```

```
    """Split a trim correction into a leading STFT and a lagging/holding LTFT.

    During the ramp STFT tracks the ramp (leads) while LTFT lags at half-weight.
    Once the fault is "developed" (ramp has reached 1.0) STFT decays toward 0
    over ``handoff_tau`` rows while LTFT integrates up to the full offset and
    holds it.

    Parameters
    ----------
    ramp : np.ndarray
        The 0→1 ramp from ``_build_ramp`` (0 pre-onset, 1 post-ramp).
    stft_peak, ltft_full : np.ndarray
        Per-row peak STFT delta and full LTFT offset (may already be
        idle-weighted by the caller).
    handoff_tau : float
        Rows over which STFT bleeds to ~0 after the ramp completes.

    Returns
    -------
    (stft_mean_delta, ltft_delta) : tuple[np.ndarray, np.ndarray]
        Mean (noise-free) trim deltas to add to the baseline trims.  The
        caller adds oscillation noise to STFT separately so it keeps
        wandering around its decaying mean.
    """
    n = len(ramp)
    # First row where the fault is fully developed (ramp == 1.0).
    if np.any(ramp >= 1.0):
        dev_start = int(np.argmax(ramp >= 1.0))
    else:
        dev_start = n  # ramp never completes → no developed region
    t_since_dev = np.maximum(0.0, np.arange(n) - dev_start)
    p = np.clip(t_since_dev / handoff_tau, 0.0, 1.0)  # 0 at ramp end → 1 after tau

    stft_mean = stft_peak * ramp * (1.0 - p)          # leads, then bleeds to 0
    ltft = ltft_full * (0.5 * ramp + 0.5 * p)          # lags, then climbs to full
    return stft_mean, ltft


@dataclass
class InjectionParams:
    """Complete, reproducible specification of one fault injection.

    Stored in the output DataFrame's ``attrs["injection"]`` so any result
    can be traced back to exactly how it was produced.
    """

    fault_type: FaultType
    onset_idx: int        # row index where ramp begins
    ramp_len: int         # rows over which fault grows from 0 → full magnitude
    magnitude: float      # fault-specific unit: kPa / % / °C-target / ratio
    noise_std: float      # Gaussian σ multiplier added to injected signal (0 =
        ↪ deterministic)
    random_seed: int | None


# --- Public API ---------------------------------------------------------------

def inject_fault(df: pd.DataFrame, params: InjectionParams) -> pd.DataFrame:
    """Return a copy of *df* with one fault injected according to *params*.
```

```
    The original DataFrame is never modified. The returned copy carries two
    extra ``attrs`` keys:
    - ``"fault_label"``  ---the fault_type string (for labelling training data)
    - ``"injection"``    ---the InjectionParams used (for reproducibility)

    All injected values are clamped to physical bounds before writing.
    """
    out = df.copy()
    n = len(out)
    rng = np.random.default_rng(params.random_seed)
    ramp = _build_ramp(n, params.onset_idx, params.ramp_len)

    def noise(scale: float) -> np.ndarray:
        if params.noise_std == 0.0 or scale == 0.0:
            return np.zeros(n)
        return rng.normal(0.0, params.noise_std * scale, size=n)

    dispatch: dict[str, Callable] = {
        "air_system": _inject_air_system,
        "fuel_system": _inject_fuel_system,
        "coolant_temp_sensor": _inject_coolant_temp,
        "throttle_position_sensor": _inject_tps,
    }
    if params.fault_type not in dispatch:
        raise ValueError(
            f"Unknown fault_type {params.fault_type!r}. "
            f"Valid options: {list(dispatch)}"
        )

    dispatch[params.fault_type](out, ramp, params.magnitude, noise)
    out.attrs = {**df.attrs, "injection": params, "fault_label": params.fault_type}
    return out


def inject_session(
    df: pd.DataFrame,
    fault_type: FaultType,
    *,
    mode: Literal["ramp", "step"] = "ramp",
    onset_fraction: float = 0.40,
    ramp_fraction: float = 0.15,
    magnitude: float | None = None,
    noise_std: float = 0.3,
    random_seed: int | None = None,
) -> pd.DataFrame:
    """Convenience wrapper: inject a fault using session-relative fractions.

    Translates ``onset_fraction`` and ``ramp_fraction`` into row indices so
    callers don't need to know the session length in advance.

    Parameters
    ----------
    df : pd.DataFrame
        Clean session from ``load_carobd_csv``.
    fault_type : FaultType
        One of the 4 supported fault strings.
    mode : {"ramp", "step"}
        "ramp"---fault degrades linearly over ``ramp_fraction`` of the session.
```

```
        "step"---fault is fully developed one row after onset (``ramp_len = 1``).
        Physics clamps (e.g. 1 °C/s coolant inertia) still apply in step mode.
    onset_fraction : float
        Fraction of session rows before fault onset (default 0.40 = 40 %).
        A pre-fault baseline period helps the classifier distinguish healthy
        from early-fault windows.
    ramp_fraction : float
        Fraction of session rows used for the ramp-up (default 0.15 = 15 %).
        Ignored when mode="step".
    magnitude : float or None
        Fault-specific magnitude. Falls back to ``_DEFAULT_MAGNITUDE`` if None.
    noise_std : float
        Gaussian σ multiplier for injected signals. 0 = deterministic.
    random_seed : int or None
        Seed for reproducibility.

    Returns
    -------
    pd.DataFrame
        Injected session with ``attrs["fault_label"]`` and ``attrs["injection"]``.
    """
    n = len(df)
    onset = int(n * onset_fraction)
    ramp_len = 1 if mode == "step" else max(1, int(n * ramp_fraction))
    mag = magnitude if magnitude is not None else _DEFAULT_MAGNITUDE[fault_type]

    params = InjectionParams(
        fault_type=fault_type,
        onset_idx=onset,
        ramp_len=ramp_len,
        magnitude=mag,
        noise_std=noise_std,
        random_seed=random_seed,
    )
    return inject_fault(df, params)


# --- Ramp helper ------------------------------------------------------------

def _build_ramp(n_rows: int, onset: int, ramp_len: int) -> np.ndarray:
    """0.0 before onset, linearly 0→1 over ramp_len rows, 1.0 afterwards."""
    t = np.zeros(n_rows, dtype=float)
    ramp_end = min(onset + ramp_len, n_rows)
    if onset < n_rows:
        steps = ramp_end - onset
        t[onset:ramp_end] = np.linspace(0.0, 1.0, steps, endpoint=False)
    if ramp_end < n_rows:
        t[ramp_end:] = 1.0
    return t


# --- Per-fault injectors ----------------------------------------------------

def _inject_air_system(
    df: pd.DataFrame,
    ramp: np.ndarray,
    magnitude_kpa: float,
    noise: Callable[[float], np.ndarray],
) -> None:
```

```
    """Speed-density vacuum leak.

    carOBD is a MAP-based (speed-density) engine---there is no MAF PID.  On
    such engines a vacuum leak does NOT produce a large fuel-trim swing: the
    MAP sensor measures the post-leak manifold pressure directly, so the ECU
    re-computes airflow from the higher MAP and largely self-compensates fuel.
    The robust, observable signature is mechanical:

      PRIMARY   ENGINE_RPM rises at idle (leak == partly-open throttle blade)
      PRIMARY   INTAKE_MANIFOLD_PRESSURE rises slightly at idle (less vacuum)
      secondary ENGINE_LOAD rises slightly (higher calculated load)
      small     a tiny, idle-only positive STFT bump that washes out off-idle
                and hands off to a marginal LTFT (P0-3 handoff)

    Off-idle (load high, RPM/speed up) every effect decays toward zero---a
    vacuum leak is a fraction of total airflow at idle but negligible at WOT.

    Clamps: MAP ≤ barometric (naturally-aspirated); idle RPM ≤ ceiling;
    STFT/LTFT within OBD-II ±25 %.
    """
    baro = (
        df["ABSOLUTE_BAROMETRIC_PRESSURE"].to_numpy(dtype=float)
        if "ABSOLUTE_BAROMETRIC_PRESSURE" in df.columns
        else np.full(len(df), _BARO_FALLBACK)
    )

    # Sharper idle-weight than the old 1−load/60: the effect genuinely vanishes
    # off-idle (clip to 0, not 0.3) because on a speed-density engine the leak's
    # contribution to airflow is negligible once the throttle is open.
    load = df["ENGINE_LOAD"].to_numpy(dtype=float)
    idle_weight = np.clip(1.0 - load / 40.0, 0.0, 1.0)

    # -- PRIMARY 1: idle RPM bump ------------------------------------------
    # Only at genuine idle (low RPM AND ~stationary); the leak pulls the idle
    # speed up like a cracked-open throttle.  Clamp to a sane idle ceiling.
    rpm = df["ENGINE_RPM"].to_numpy(dtype=float)
    speed = (
        df["VEHICLE_SPEED"].to_numpy(dtype=float)
        if "VEHICLE_SPEED" in df.columns
        else np.zeros(len(df))
    )
    idle_mask = (rpm < _AIR_IDLE_RPM_MAX) & (speed < _AIR_IDLE_SPEED_MAX)
    rpm_bump = ramp * idle_weight * magnitude_kpa * _AIR_RPM_PER_KPA + noise(2.0)
    rpm_new = np.where(idle_mask, np.minimum(rpm + rpm_bump, _AIR_IDLE_RPM_CEILING), rpm)
    df["ENGINE_RPM"] = np.clip(rpm_new, a_min=0.0, a_max=None)

    # -- PRIMARY 2: slightly elevated MAP at idle --------------------------
    map_delta = ramp * magnitude_kpa * idle_weight * _AIR_MAP_COEFF + noise(0.3) *
        ↪ idle_weight
    df["INTAKE_MANIFOLD_PRESSURE"] = np.clip(
        df["INTAKE_MANIFOLD_PRESSURE"].to_numpy(dtype=float) + map_delta,
        a_min=0.0,
        a_max=baro,
    )

    # -- secondary: calculated load rises a little at idle -----------------
    load_delta = ramp * magnitude_kpa * idle_weight * _AIR_LOAD_COEFF + noise(0.2) *
        ↪ idle_weight
    df["ENGINE_LOAD"] = np.clip(load + load_delta, 0.0, 100.0)
```

```
335
336     # -- small idle-only fuel-trim bump with STFT→LTFT handoff (P0-3) ---------
337     # Tiny coefficients, idle-weighted so the bump washes out off-idle.  The
338     # handoff makes the developed-state STFT decay toward 0 while a marginal
339     # LTFT holds---matching real speed-density behaviour where any trim is
340     # small and transient.
341     stft_peak = magnitude_kpa * idle_weight * _AIR_STFT_COEFF
342     ltft_full = magnitude_kpa * idle_weight * _AIR_LTFT_COEFF
343     stft_mean, ltft_delta = _steady_state_trim(ramp, stft_peak, ltft_full)
344     df["SHORT_TERM_FUEL_TRIM_BANK_1"] = np.clip(
345         df["SHORT_TERM_FUEL_TRIM_BANK_1"].to_numpy(dtype=float) + stft_mean + noise(0.3),
346         -_STFT_MAX,
347         _STFT_MAX,
348     )
349     df["LONG_TERM_FUEL_TRIM_BANK_1"] = np.clip(
350         df["LONG_TERM_FUEL_TRIM_BANK_1"].to_numpy(dtype=float) + ltft_delta +
            ↪ noise(0.05),
351         -_LTFT_MAX,
352         _LTFT_MAX,
353     )
354
355
356 def _inject_fuel_system(
357     df: pd.DataFrame,
358     ramp: np.ndarray,
359     magnitude_pct: float,
360     noise: Callable[[float], np.ndarray],
361 ) -> None:
362     """Injector clog / low fuel rail pressure.
363
364     Less fuel per spray cycle → chronic lean condition → ECU compensates with
365     a rising LTFT. MAP is intentionally left unchanged---that unchanged MAP is
366     the key feature distinguishing this fault from an air-system fault.
367
368     STFT→LTFT handoff (P0-3): during the ramp STFT leads (rises first) while
369     LTFT lags; once the fault is developed, STFT bleeds back toward 0 (still
370     oscillating on noise) and LTFT holds the chronic offset.  A developed
371     fuel-system fault therefore shows high LTFT and near-zero MEAN STFT---the
372     relationship a real adaptive ECU produces.
373     """
374     # STFT leads at ~half the offset during the transient, then decays to 0;
375     # LTFT lags then climbs to the full offset and holds it.
376     stft_peak = np.full(len(ramp), magnitude_pct * 0.5)
377     ltft_full = np.full(len(ramp), magnitude_pct)
378     stft_mean, ltft_delta = _steady_state_trim(ramp, stft_peak, ltft_full)
379
380     df["LONG_TERM_FUEL_TRIM_BANK_1"] = np.clip(
381         df["LONG_TERM_FUEL_TRIM_BANK_1"].to_numpy(dtype=float) + ltft_delta + noise(0.2),
382         -_LTFT_MAX,
383         _LTFT_MAX,
384     )
385     # STFT keeps its oscillation noise around the decaying mean.
386     df["SHORT_TERM_FUEL_TRIM_BANK_1"] = np.clip(
387         df["SHORT_TERM_FUEL_TRIM_BANK_1"].to_numpy(dtype=float) + stft_mean + noise(1.0),
388         -_STFT_MAX,
389         _STFT_MAX,
390     )
391     # MAP deliberately unchanged (differentiator from air_system)
392
```

```
    # ENGINE_LOAD deliberately unchanged: the PID is normalised AIRFLOW, and a
    # clogged injector cuts fuel, not air.  (A real driver compensating for
    # lost torque would RAISE it---driver behaviour is outside replayed data.)

    # ENGINE_RPM: injector clog causes idle roughness---small low-frequency jitter
    # only at low RPM (< 1200 rpm) where cylinder-dropout effects are felt.
    rpm = df["ENGINE_RPM"].to_numpy(dtype=float)
    idle_mask = rpm < 1200.0
    if idle_mask.any():
        rpm_jitter = noise(15.0) * ramp  # up to ±4.5 rpm σ at full ramp
        df["ENGINE_RPM"] = np.clip(
            rpm + np.where(idle_mask, rpm_jitter, 0.0),
            a_min=0.0,
            a_max=None,
        )


def _inject_coolant_temp(
    df: pd.DataFrame,
    ramp: np.ndarray,
    stuck_temp_c: float,
    noise: Callable[[float], np.ndarray],
) -> None:
    """Stuck / biased ECT sensor.

    The sensor reading drifts toward a stuck (too-cold) value. The ECU thinks
    the engine never reached operating temperature and retards ignition timing.

    The 1 °C/sec max-change constraint is enforced sample-by-sample after
    computing the target trajectory, preserving thermal inertia realism.

    Rich-bias signature (P1-2): the ECU holds cold-enrichment fuelling because
    it believes the engine is cold, so once closed-loop is reached the fuel
    trims go NEGATIVE (rich).  This is the dominant OBD-detectable consequence
    and is modelled here with the same STFT→LTFT handoff as the fuel-system
    fault, alongside the retarded timing.
    """
    actual = df["COOLANT_TEMPERATURE"].to_numpy(dtype=float)

    # Target: drift from current reading toward stuck_temp_c
    raw_target = actual + ramp * (stuck_temp_c - actual) + noise(0.3)

    # Enforce thermal inertia: no more than 1 °C change per second
    target = raw_target.copy()
    for i in range(1, len(target)):
        delta = target[i] - target[i - 1]
        target[i] = target[i - 1] + np.clip(delta, -_COOLANT_MAX_DELTA,
            ↪ _COOLANT_MAX_DELTA)

    df["COOLANT_TEMPERATURE"] = np.clip(target, _COOLANT_MIN, _COOLANT_MAX)

    # TIMING_ADVANCE: ECU retards timing proportional to perceived cold deficit.
    # Normal operating temp ≈ 90 °C; max retard ≈ 5° at full fault.
    normal_temp = 90.0
    temp_deficit = np.maximum(0.0, normal_temp -
        ↪ df["COOLANT_TEMPERATURE"].to_numpy(dtype=float))
    timing_retard = ramp * np.minimum(temp_deficit * 0.1, 5.0)
    df["TIMING_ADVANCE"] = np.clip(
        df["TIMING_ADVANCE"].to_numpy(dtype=float) - timing_retard + noise(0.1),
```

```
        _TIMING_MIN,
        _TIMING_MAX,
    )

    # Rich fuel-trim bias (P1-2): negative STFT/LTFT from chronic cold-enrichment,
    # with the same STFT-leads-then-hands-off-to-LTFT dynamics as a fuel fault.
    stft_peak = np.full(len(ramp), _COOLANT_RICH_STFT_PEAK)
    ltft_full = np.full(len(ramp), _COOLANT_RICH_LTFT)
    stft_mean, ltft_delta = _steady_state_trim(ramp, stft_peak, ltft_full)
    df["SHORT_TERM_FUEL_TRIM_BANK_1"] = np.clip(
        df["SHORT_TERM_FUEL_TRIM_BANK_1"].to_numpy(dtype=float) + stft_mean + noise(0.5),
        -_STFT_MAX,
        _STFT_MAX,
    )
    df["LONG_TERM_FUEL_TRIM_BANK_1"] = np.clip(
        df["LONG_TERM_FUEL_TRIM_BANK_1"].to_numpy(dtype=float) + ltft_delta + noise(0.1),
        -_LTFT_MAX,
        _LTFT_MAX,
    )


def _inject_tps(
    df: pd.DataFrame,
    ramp: np.ndarray,
    drift_factor: float,
    noise: Callable[[float], np.ndarray],
) -> None:
    """TPS potentiometer wear.

    The THROTTLE reading drifts upward relative to the actual pedal position.
    Only active when ACCELERATOR_PEDAL_POSITION_D > 10 % (idle rows are clean
   ---the potentiometer is not loaded at rest).

    COMMANDED_THROTTLE_ACTUATOR is left unchanged so the commanded-vs-reported
    divergence is a visible second feature for the classifier.
    """
    pedal = df["ACCELERATOR_PEDAL_POSITION_D"].to_numpy(dtype=float)
    throttle = df["THROTTLE"].to_numpy(dtype=float)

    effective_factor = 1.0 + ramp * (drift_factor - 1.0)
    injected = throttle * effective_factor + noise(0.2)

    # Only apply drift where pedal is engaged; leave idle rows unaffected
    pedal_active = pedal > 10.0
    df["THROTTLE"] = np.clip(
        np.where(pedal_active, injected, throttle),
        _THROTTLE_MIN,
        _THROTTLE_MAX,
    )
```

Listing 1: `src/injection/fault_injector.py` — the physics-constrained fault injection engine.

```python
"""XGBoost fault classifier --- Week 4 model.

Why XGBoost after Random Forest?
--------------------------------
RF grows trees independently and averages them --- every tree is equally
weighted. XGBoost grows trees sequentially: each new tree focuses on the
windows the previous trees got wrong (gradient boosting). On tabular
sensor data, boosting typically squeezes out another 2--5 % F1 over
bagging --- worth having as the model we ship.

More importantly for this project: XGBoost + TreeExplainer is the
gold-standard combination for SHAP explanations. The SHAP values from
XGBoost are computed exactly (not sampled), so the explanation
"LTFT pushed this prediction toward fuel_system by 0.43" is precise
rather than approximate.

This module mirrors the RF module's API: session_split lives in
classifier.py and is shared by both models.
"""

from __future__ import annotations

import json
import logging
import pickle
from pathlib import Path

import numpy as np
import pandas as pd
import xgboost as xgb
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.utils.class_weight import compute_sample_weight

from src.config import MODELS_DIR, RANDOM_SEED, RESULTS_DIR
from src.features.dataset_builder import FAULT_TYPES, LABEL_TO_ID
from src.features.normalizer import BaselineNormalizer, normalised_feature_names
from src.models.classifier import ALL_LABELS

log = logging.getLogger(__name__)

# fuel_system over-fires (precision 0.457 at plan time): down-weight its
# samples so the decision boundary retreats toward higher-confidence windows.
_FUEL_WEIGHT = 0.5


def train(
    train_df: pd.DataFrame,
    *,
    n_estimators: int = 300,
    max_depth: int = 6,
    learning_rate: float = 0.1,
    subsample: float = 0.8,
    colsample_bytree: float = 0.8,
    random_seed: int = RANDOM_SEED,
    class_balance: bool = True,
    fuel_downweight: float = _FUEL_WEIGHT,
) -> tuple[xgb.XGBClassifier, BaselineNormalizer]:
    """Fit a normaliser then an XGBoost classifier on *train_df*.
```

```
59
60      Returns both the fitted model and the fitted normaliser so they
61      can be saved together and used as a unit during inference.
62
63      Parameters
64      ----------
65      train_df : pd.DataFrame
66          Training split from ``session_split``.
67      class_balance : bool, default True
68          If True, weight each sample inversely to its CLASS frequency
69          (sklearn "balanced"). After the 129-file expansion the 6 fault
70          classes are uneven---cold_start (~5% of windows) and healthy are
71          scarcer than the 4 injected faults---so this stops the model being
72          dominated by the majority classes. NOTE: this balances CLASSES, not
73          operating regimes; the accel-regime scarcity is a window-length
74          problem and is not addressed here.
75      fuel_downweight : float, default 0.5
76          Extra multiplier on fuel_system samples (historically over-fired,
77          precision 0.457). WARNING: 0.5 was tuned on the pre-fix 8-file
78          dataset; revisit it in the ablation now that we train on 129 files.
79          Set to 1.0 to disable.
80
81      Returns
82      -------
83      (XGBClassifier, BaselineNormalizer)
84      """
85      norm = BaselineNormalizer()
86      train_norm = norm.fit_transform(train_df)
87
88      feat_cols = normalised_feature_names()
89      X = train_norm[feat_cols].to_numpy(dtype=float)
90      y = train_norm["label_id"].to_numpy(dtype=int)
91
92      # Base weights: balance the 6 classes (rescues scarce cold_start/healthy).
93      if class_balance:
94          sample_weights = compute_sample_weight("balanced", y)
95      else:
96          sample_weights = np.ones(len(y), dtype=float)
97
98      # fuel_system down-weighted ON TOP of class balance to pull its decision
99      # boundary back from the healthy/air/TPS region. Stale constant---see docstring.
100     if fuel_downweight != 1.0:
101         fuel_id = LABEL_TO_ID["fuel_system"]
102         sample_weights = sample_weights * np.where(y == fuel_id, fuel_downweight, 1.0)
103
104     clf = xgb.XGBClassifier(
105         n_estimators=n_estimators,
106         max_depth=max_depth,
107         learning_rate=learning_rate,
108         subsample=subsample,
109         colsample_bytree=colsample_bytree,
110         objective="multi:softprob",
111         num_class=len(ALL_LABELS),
112         eval_metric="mlogloss",
113         random_state=random_seed,
114         n_jobs=-1,
115         verbosity=0,
116     )
117     clf.fit(X, y, sample_weight=sample_weights)
```

```
    log.info(
        "Trained XGB: %d trees, depth %d, lr %.3f, %d train samples, %d features "
        "(class_balance=%s, fuel_downweight=%.2f)",
        n_estimators,
        max_depth,
        learning_rate,
        len(X),
        len(feat_cols),
        class_balance,
        fuel_downweight,
    )
    return clf, norm


def evaluate(
    clf: xgb.XGBClassifier,
    norm: BaselineNormalizer,
    test_df: pd.DataFrame,
) -> dict:
    """Evaluate XGBoost on the test split.

    Parameters
    ----------
    clf : XGBClassifier
    norm : BaselineNormalizer
        Must be the same normaliser used during training.
    test_df : pd.DataFrame

    Returns
    -------
    dict with keys: macro_f1, per_class, confusion_matrix, test_sessions
    """
    test_norm = norm.transform(test_df)
    feat_cols = normalised_feature_names()
    X_test = test_norm[feat_cols].to_numpy(dtype=float)
    y_true = test_df["label_id"].to_numpy(dtype=int)
    y_pred = clf.predict(X_test)

    report = classification_report(
        y_true,
        y_pred,
        labels=list(range(len(ALL_LABELS))),
        target_names=ALL_LABELS,
        output_dict=True,
        zero_division=0,
    )
    cm = confusion_matrix(y_true, y_pred, labels=list(range(len(ALL_LABELS))))
    macro_f1 = report["macro avg"]["f1-score"]
    log.info("XGB test macro-F1: %.4f", macro_f1)

    return {
        "macro_f1": macro_f1,
        "per_class": {
            label: {
                "precision": report[label]["precision"],
                "recall": report[label]["recall"],
                "f1": report[label]["f1-score"],
                "support": report[label]["support"],
            }
```

```
            for label in ALL_LABELS
        },
        "confusion_matrix": cm.tolist(),
        "label_order": ALL_LABELS,
        "test_sessions": sorted(test_df["session_id"].unique().tolist()),
    }


def save_model(
    clf: xgb.XGBClassifier,
    norm: BaselineNormalizer,
    results: dict,
    models_dir: Path | None = None,
    results_dir: Path | None = None,
) -> Path:
    """Save model, normaliser, and results to disk."""
    models_dir = Path(models_dir or MODELS_DIR)
    results_dir = Path(results_dir or RESULTS_DIR)
    models_dir.mkdir(parents=True, exist_ok=True)
    results_dir.mkdir(parents=True, exist_ok=True)

    model_path = models_dir / "xgb_classifier_v1.pkl"
    with open(model_path, "wb") as f:
        pickle.dump({"model": clf, "normalizer": norm}, f)

    results_path = results_dir / "xgb_classifier_v1_results.json"
    with open(results_path, "w") as f:
        json.dump(results, f, indent=2)

    log.info("XGB model saved to %s", model_path)
    return model_path


def load_model(models_dir: Path | None = None) -> tuple[xgb.XGBClassifier,
    ↪ BaselineNormalizer]:
    """Load a previously saved XGBoost model and normaliser."""
    models_dir = Path(models_dir or MODELS_DIR)
    path = models_dir / "xgb_classifier_v1.pkl"
    if not path.exists():
        raise FileNotFoundError(f"No saved XGB model at {path}.")
    with open(path, "rb") as f:
        bundle = pickle.load(f)
    return bundle["model"], bundle["normalizer"]
```

Listing 2: `src/models/xgb_classifier.py` — XGBoost training, evaluation, and persistence, including class-balance weights and the fuel-system down-weight.

```
"""End-of-read engine-health report for CSV replay (self-baselined).

Design (all four decisions locked with the team):

1. Self-baseline: the first 20% of the recording (by elapsed time) is assumed
   healthy and used only to define "normal". It is EXCLUDED from the verdict---
   scoring baseline windows against themselves is circular.
2. Severity comes from the physics `compute_severity` per window (carried on
   DashboardState.severities), NOT the forecaster---the forecaster extrapolates
   off-distribution on healthy data and produced phantom 17%/27% severities.
3. Baseline-period windows are excluded from all verdict/severity aggregation.
4. If too few evaluable post-baseline windows remain, the report says
   "INSUFFICIENT DATA" rather than inventing a health verdict.

Honest scope (must appear on the rendered report):
  * This detects a fault that DEVELOPS during the recording (deviation from the
    car's own early-session healthy baseline).
  * It CANNOT detect a fault present from key-on---that becomes the baseline and
    reads as normal.
  * It assumes the first 20% is healthy. A car already faulty at key-on poisons
    the baseline.
"""

from __future__ import annotations

from dataclasses import dataclass, field

# Faults that carry a physics severity (the 4 injectable faults). healthy /
# cold_start have no severity.
SEVERITY_FAULTS = (
    "air_system",
    "fuel_system",
    "coolant_temp_sensor",
    "throttle_position_sensor",
)

# A fault is "Detected" if it is the winning label in at least this fraction of
# evaluable post-baseline windows (mirrors the score_recording verdict gate).
_DETECT_FRACTION = 0.40
# Below this many evaluable post-baseline windows, refuse to give a verdict.
_MIN_EVAL_WINDOWS = 10


@dataclass
class FaultReport:
    fault: str
    status: str  # "detected" | "healthy" | "untested"
    severity_pct: float | None  # 0..100, only for detected+evaluable; else None
    window_share_pct: float  # % of evaluable windows that got this label


@dataclass
class SessionReport:
    verdict: str
    n_windows_total: int
    n_baseline_windows: int
    n_evaluable_windows: int
    n_untested_windows: int
```

```
    faults: list[FaultReport] = field(default_factory=list)
    caveats: list[str] = field(default_factory=list)


def _percentile(values: list[float], pct: float) -> float:
    """Linear-interpolated percentile (avoids a numpy dependency here)."""
    if not values:
        return 0.0
    s = sorted(values)
    if len(s) == 1:
        return s[0]
    k = (len(s) - 1) * (pct / 100.0)
    lo = int(k)
    hi = min(lo + 1, len(s) - 1)
    return s[lo] + (s[hi] - s[lo]) * (k - lo)


def build_session_report(
    records: list[dict],
    untested_fault_set: set[str],
    *,
    baseline_frac: float = 0.20,
    detect_fraction: float = _DETECT_FRACTION,
    min_eval_windows: int = _MIN_EVAL_WINDOWS,
) -> SessionReport:
    """Build the end-of-read report.

    Parameters
    ----------
    records : list of dict
        One entry per scored window, in time order, each with:
          "elapsed_s": float
          "label":     str   (winning classifier label)
          "severities": dict[str, float]  (physics severity 0..1 per fault)
        Baseline windows are identified here by elapsed_s, so pass ALL windows.
    untested_fault_set : set[str]
        Faults whose primary PID is unavailable on this vehicle (from
        src.eval.pid_availability.untested_faults). These are never Detected;
        they are reported "untested".
    """
    n_total = len(records)
    if n_total == 0:
        return SessionReport("INSUFFICIENT DATA (no windows)", 0, 0, 0, 0)

    # Baseline cut-off by elapsed time (first `baseline_frac` of the recording).
    t_max = max(r["elapsed_s"] for r in records)
    cutoff = t_max * baseline_frac
    post = [r for r in records if r["elapsed_s"] >= cutoff]
    n_baseline = n_total - len(post)

    # Untested windows (winning label is an untested fault) are set aside.
    evaluable = [r for r in post if r["label"] not in untested_fault_set]
    n_untested = len(post) - len(evaluable)
    n_eval = len(evaluable)

    caveats = [
        "Detects faults that DEVELOP during the recording; cannot detect a fault "
        "present from key-on (it becomes the baseline).",
        f"Assumes the first {int(baseline_frac * 100)}% of the recording is healthy.",
```

```
    ]

    # Untested faults are reported regardless of how much evaluable data exists---
    # "we can't test this on this vehicle" is information even when the rest is thin.
    untested_reports = [
        FaultReport(f, "untested", None, 0.0)
        for f in SEVERITY_FAULTS
        if f in untested_fault_set
    ]

    if n_eval < min_eval_windows:
        return SessionReport(
            verdict=f"INSUFFICIENT DATA ({n_eval} evaluable windows; need >=
                ↪ {min_eval_windows})",
            n_windows_total=n_total,
            n_baseline_windows=n_baseline,
            n_evaluable_windows=n_eval,
            n_untested_windows=n_untested,
            faults=untested_reports,
            caveats=caveats,
        )

    # Per-fault status over evaluable post-baseline windows.
    faults: list[FaultReport] = list(untested_reports)
    any_detected = False
    for fault in SEVERITY_FAULTS:
        if fault in untested_fault_set:
            continue  # already reported as untested
        hits = [r for r in evaluable if r["label"] == fault]
        share = 100.0 * len(hits) / n_eval
        if share >= detect_fraction * 100.0:
            sev_vals = [r["severities"].get(fault, 0.0) for r in evaluable]
            sev_pct = round(_percentile(sev_vals, 95.0) * 100.0, 1)
            if sev_pct < 1.0:
                # Label dominance with ~zero physical severity is the signature
                # of a missing-PID / feature artifact, not a physical fault---
                # report it as inconclusive rather than "DETECTED, severity 0%".
                faults.append(FaultReport(fault, "inconclusive", None, round(share, 1)))
            else:
                faults.append(FaultReport(fault, "detected", sev_pct, round(share, 1)))
                any_detected = True
        else:
            faults.append(FaultReport(fault, "healthy", None, round(share, 1)))

    if any_detected:
        verdict = "DEVELOPING FAULT(S) DETECTED"
    else:
        verdict = "HEALTHY (no developing fault within session)"

    return SessionReport(
        verdict=verdict,
        n_windows_total=n_total,
        n_baseline_windows=n_baseline,
        n_evaluable_windows=n_eval,
        n_untested_windows=n_untested,
        faults=faults,
        caveats=caveats,
    )
```

Listing 3: `src/eval/session_report.py` — the end-of-read health report: 20 % self-baseline, physics-severity 95th percentile, inconclusive rule, and the minimum-evaluable-windows floor.

```python
"""Per-fault PID requirements and evaluability for cross-vehicle scoring.

Different ECUs expose different PIDs. When a car does not report a PID that a
fault's PRIMARY signal depends on, that fault cannot be honestly scored on that
car: the model still emits a class (XGBoost routes NaN features to a default
split), but the number is meaningless. This module marks such faults as
"Untested" instead of letting a fake score through.

It also reports which USEFUL_PIDS are unavailable at all, because a missing PID
feeds features used across MANY classes (e.g. INTAKE_MANIFOLD_PRESSURE feeds the
MAP_PER_THROTTLE ratio and MAP statistics). Its absence therefore reduces
confidence in EVERY score, not only the fault that names it as primary.

A PID is "available" iff its column exists AND has at least one non-NaN value---
so a present-but-entirely-empty column (the Yaris MAP case) counts as missing.
"""

from __future__ import annotations

import pandas as pd

from src.config import USEFUL_PIDS

# A fault is evaluable only if ALL of its required PIDs are available. "Required"
# means the PID(s) carrying the fault's primary discriminative signal, per the
# injector's fault definitions in src/injection/fault_injector.py.
FAULT_REQUIRED_PIDS: dict[str, set[str]] = {
    "healthy": set(),  # baseline---always evaluable
    "cold_start": {"COOLANT_TEMPERATURE"},
    "air_system": {"INTAKE_MANIFOLD_PRESSURE"},  # speed-density signature dies without
        ↪ MAP
    "fuel_system": {"LONG_TERM_FUEL_TRIM_BANK_1"},  # LTFT is the primary fuel signal
    "coolant_temp_sensor": {"COOLANT_TEMPERATURE"},
    # TPS detection rests on reported-vs-commanded divergence AND the
    # THROTTLE_TO_PEDAL_RATIO feature---the pedal channels are load-bearing.
    # A car missing pedals gets a fabricated ratio, which classifies as a
    # phantom TPS fault (observed: "DETECTED, severity 0%" on missing pedals).
    "throttle_position_sensor": {
        "THROTTLE",
        "COMMANDED_THROTTLE_ACTUATOR",
        "ACCELERATOR_PEDAL_POSITION_D",
        "ACCELERATOR_PEDAL_POSITION_E",
    },
}


def available_pids(df: pd.DataFrame) -> set[str]:
    """USEFUL_PIDS present in *df* AND carrying at least one non-NaN value."""
    return {p for p in USEFUL_PIDS if p in df.columns and df[p].notna().any()}


def missing_pids(df: pd.DataFrame) -> list[str]:
    """USEFUL_PIDS absent or entirely NaN in this recording (contract order)."""
    avail = available_pids(df)
    return [p for p in USEFUL_PIDS if p not in avail]


def untested_faults(avail: set[str]) -> dict[str, list[str]]:
```

```
    """Map each fault that CANNOT be evaluated -> the required PIDs it lacks."""
    out: dict[str, list[str]] = {}
    for fault, required in FAULT_REQUIRED_PIDS.items():
        lacking = sorted(required - avail)
        if lacking:
            out[fault] = lacking
    return out


def evaluable_faults(avail: set[str]) -> list[str]:
    """Faults whose required PIDs are all available."""
    return [fault for fault, req in FAULT_REQUIRED_PIDS.items() if req <= avail]
```

Listing 4: `src/eval/pid_availability.py` — per-fault required PIDs and the Untested contract.

B Key Source Code Listings — OBD-LOG Android App

The two listings below are copied from the OBD-LOG repository (`github.com/Terminatorbds/OBD-LOG`) from `app/src/main/java/com/example/obdlogger/`. `ObdPids.kt` defines the tiered PID table and the reply parser; `ObdLogger.kt` performs ELM327 initialization, ECU capability detection via the support bitmasks, and single-PID reads.

```
1  package com.example.obdlogger
2
3  // One OBD-II PID definition: command we send, response prefix a valid reply
4  // starts with, data-byte count, unit, a fault-relevance tier, and the decode
5  // formula. `a`/`b` are the first/second data bytes.
6  //
7  // tier 1 = Core (highest engine-fault signal), 2 = Extended (useful context),
8  // 3 = Full (niche / redundant for engine faults; kept for throughput testing).
9  data class ObdPid(
10     val name: String,
11     val command: String,
12     val responsePrefix: String,
13     val pidNumber: Int,
14     val bytes: Int,
15     val unit: String,
16     val tier: Int,
17     val formula: (a: Int, b: Int) -> Double
18 )
19
20 object Pids {
21
22     // Order here = column order in the CSV (all 23 are always written; mode only
23     // changes which are actually polled/filled).
24     val ALL: List<ObdPid> = listOf(
25         // --- Tier 1: Core fault signals ---
26         ObdPid("ENGINE_RPM",                    "010C", "410C", 0x0C, 2, "rpm",   1)
               ↪ { a, b -> (a * 256 + b) / 4.0 },
27         ObdPid("ENGINE_LOAD",                   "0104", "4104", 0x04, 1, "%",     1)
               ↪ { a, _ -> a * 100.0 / 255 },
28         ObdPid("INTAKE_MANIFOLD_PRESSURE",      "010B", "410B", 0x0B, 1, "kPa",   1)
               ↪ { a, _ -> a.toDouble() },
29         // Added after your Yaris returned no MAP (010B). MAF is the real airflow
30         // channel on a MAF-based car. This is a 24th column beyond your 23-PID
31         // training set---delete this line to keep the schema frozen at 23.
32         ObdPid("MASS_AIR_FLOW",                 "0110", "4110", 0x10, 2, "g/s",   1)
               ↪ { a, b -> (a * 256 + b) / 100.0 },
33         ObdPid("COOLANT_TEMPERATURE",           "0105", "4105", 0x05, 1, "C",     1)
               ↪ { a, _ -> a - 40.0 },
34         ObdPid("INTAKE_AIR_TEMP",               "010F", "410F", 0x0F, 1, "C",     1)
               ↪ { a, _ -> a - 40.0 },
35         ObdPid("SHORT_TERM_FUEL_TRIM_BANK_1",   "0106", "4106", 0x06, 1, "%",     1)
               ↪ { a, _ -> (a - 128) * 100.0 / 128 },
36         ObdPid("LONG_TERM_FUEL_TRIM_BANK_1",    "0107", "4107", 0x07, 1, "%",     1)
               ↪ { a, _ -> (a - 128) * 100.0 / 128 },
37         ObdPid("TIMING_ADVANCE",                "010E", "410E", 0x0E, 1, "deg",   1)
               ↪ { a, _ -> a / 2.0 - 64 },
38
```

```
        // --- Tier 2: Extended context ---
        ObdPid("VEHICLE_SPEED",                    "010D", "410D", 0x0D, 1, "km/h",  2) { a, _ -> a.toDouble() },
        ObdPid("THROTTLE",                         "0111", "4111", 0x11, 1, "%",     2) { a, _ -> a * 100.0 / 255 },
        ObdPid("FUEL_AIR_COMMANDED_EQUIV_RATIO",   "0144", "4144", 0x44, 2, "ratio", 2) { a, b -> (a * 256 + b) / 32768.0 },
        ObdPid("CONTROL_MODULE_VOLTAGE",           "0142", "4142", 0x42, 2, "V",     2) { a, b -> (a * 256 + b) / 1000.0 },
        ObdPid("ABSOLUTE_BAROMETRIC_PRESSURE",     "0133", "4133", 0x33, 1, "kPa",   2) { a, _ -> a.toDouble() },

        // --- Tier 3: Full (throughput test / completeness) ---
        ObdPid("ENGINE_RUN_TIME",                  "011F", "411F", 0x1F, 2, "s",     3) { a, b -> (a * 256 + b).toDouble() },
        ObdPid("FUEL_TANK",                        "012F", "412F", 0x2F, 1, "%",     3) { a, _ -> a * 100.0 / 255 },
        ObdPid("ABSOLUTE_THROTTLE_B",              "0147", "4147", 0x47, 1, "%",     3) { a, _ -> a * 100.0 / 255 },
        ObdPid("PEDAL_D",                          "0149", "4149", 0x49, 1, "%",     3) { a, _ -> a * 100.0 / 255 },
        ObdPid("PEDAL_E",                          "014A", "414A", 0x4A, 1, "%",     3) { a, _ -> a * 100.0 / 255 },
        ObdPid("COMMANDED_THROTTLE_ACTUATOR",      "014C", "414C", 0x4C, 1, "%",     3) { a, _ -> a * 100.0 / 255 },
        ObdPid("RELATIVE_THROTTLE_POSITION",       "0145", "4145", 0x45, 1, "%",     3) { a, _ -> a * 100.0 / 255 },
        ObdPid("CATALYST_TEMPERATURE_BANK1_SENSOR1","013C", "413C", 0x3C, 2, "C",     3) { a, b -> (a * 256 + b) / 10.0 - 40 },
        ObdPid("CATALYST_TEMPERATURE_BANK1_SENSOR2","013E", "413E", 0x3E, 2, "C",     3) { a, b -> (a * 256 + b) / 10.0 - 40 },
        ObdPid("COMMANDED_EVAPORATIVE_PURGE",      "012E", "412E", 0x2E, 1, "%",     3) { a, _ -> a * 100.0 / 255 },
    )

    // PIDs to poll for a given capture mode (1=Core, 2=Extended, 3=Full).
    fun forTier(maxTier: Int): List<ObdPid> = ALL.filter { it.tier <= maxTier }

    // Turn a raw ELM327 reply into a number, or null if unsupported / no data.
    fun parse(raw: String, pid: ObdPid): Double? {
        val upper = raw.uppercase()
        if (upper.contains("NO DATA") || upper.contains("ERROR") ||
            upper.contains("UNABLE") || upper.contains("STOPPED") ||
                upper.contains("?")) return null

        val clean = upper.replace(Regex("[^0-9A-F]"), "")
        val idx = clean.indexOf(pid.responsePrefix)
        if (idx < 0) return null

        val data = clean.substring(idx + pid.responsePrefix.length)
        if (data.length < pid.bytes * 2) return null

        return try {
            val a = data.substring(0, 2).toInt(16)
            val b = if (pid.bytes >= 2) data.substring(2, 4).toInt(16) else 0
            pid.formula(a, b)
        } catch (e: Exception) {
            null
        }
```

```
    }
}
```

Listing 5: `ObdPids.kt` — tiered PID definitions, decode formulas, and ELM327 reply parsing.

```
package com.example.obdlogger

// Higher-level OBD operations built on top of the raw Bluetooth link:
// initialise the ELM327, detect which PIDs the car supports, and read a PID.
class ObdLogger(private val bt: BluetoothObd) {

    // Standard ELM327 init sequence. Returns a readable log so you can see what
    // the adapter replied (useful when a clone misbehaves).
    fun initElm(): String {
        val log = StringBuilder()
        log.append("ATZ   -> ").append(bt.send("ATZ", 3000).trim()).append('\n')   // reset
        log.append("ATE0  -> ").append(bt.send("ATE0").trim()).append('\n')        // echo off (important!)
        log.append("ATL0  -> ").append(bt.send("ATL0").trim()).append('\n')        // linefeeds off
        log.append("ATS0  -> ").append(bt.send("ATS0").trim()).append('\n')        // spaces off (simpler parsing)
        log.append("ATSP0 -> ").append(bt.send("ATSP0").trim()).append('\n')       // auto-detect protocol
        log.append("0100  -> ").append(bt.send("0100", 5000).trim()).append('\n')  // wake bus / first query
        return log.toString()
    }

    // Ask the ECU which PIDs it supports via the support bitmasks, then keep only
    // the requested PIDs the car actually has. This replaces any "choose your car"
    // dropdown---the car tells us itself.
    fun detectSupported(requested: List<ObdPid>): List<ObdPid> {
        val supported = mutableSetOf<Int>()
        // 0100 -> PIDs 0x01..0x20, 0120 -> 0x21..0x40, 0140 -> 0x41..0x60
        listOf("0100" to 0x00, "0120" to 0x20, "0140" to 0x40).forEach { (cmd, base) ->
            val bits = parseSupportMask(bt.send(cmd, 3000), cmd.substring(2)) ?: return@forEach
            for (i in 0 until 32) {
                if ((bits shr (31 - i)) and 1L == 1L) supported.add(base + i + 1)
            }
        }
        // If detection failed completely, fall back to trying all requested PIDs.
        if (supported.isEmpty()) return requested
        return requested.filter { supported.contains(it.pidNumber) }
    }

    private fun parseSupportMask(raw: String, pidHex: String): Long? {
        val upper = raw.uppercase()
        if (upper.contains("NO DATA") || upper.contains("ERROR")) return null
        val clean = upper.replace(Regex("[^0-9A-F]"), "")
        val idx = clean.indexOf("41$pidHex")
        if (idx < 0) return null
        val data = clean.substring(idx + 4)
        if (data.length < 8) return null
        return try { data.substring(0, 8).toLong(16) } catch (e: Exception) { null }
    }

    // Read one PID and decode it. Null = unsupported or no data this cycle.
    fun readPid(pid: ObdPid): Double? = Pids.parse(bt.send(pid.command, 1000), pid)
}
```

Listing 6: `ObdLogger.kt` — adapter initialization and supported-PID detection.

C Full Per-Class Metrics Tables

All values below are taken directly from the evaluation artifacts in the repository's `results/` directory (`xgb_classifier_v1_results.json`, `lofo_eval.json`, `loso_cv_results.json`, `rf_classifier_v1_results.json`, `withheld_coeff_results.json`).

Table 8: XGBoost fixed-holdout (drive1 + live12) per-class metrics. Macro-F1 = 0.962.

Class	Precision	Recall	F1	Support
healthy	0.991	0.965	0.978	346
air_system	0.970	0.865	0.915	223
fuel_system	0.825	0.965	0.889	171
coolant_temp_sensor	1.000	1.000	1.000	216
throttle_position_sensor	0.979	1.000	0.990	189
cold_start	1.000	1.000	1.000	29

Table 9: Random Forest baseline, fixed holdout, earlier five-class task. Macro-F1 = 1.000 — reported as a memorization warning, not a result.

Class	Precision	Recall	F1	Support
healthy	1.000	1.000	1.000	375
air_system	1.000	1.000	1.000	163
fuel_system	1.000	1.000	1.000	163
coolant_temp_sensor	1.000	1.000	1.000	163
throttle_position_sensor	1.000	1.000	1.000	163

Table 10: LOFO cross-validation: per-class F1 for every held-out family. Mean macro-F1 = 0.977, σ = 0.012.

	Held-out family				
Class	drive	idle	live	long	ufpe
healthy	0.968	0.933	0.933	0.983	0.984
air_system	0.989	0.980	0.965	0.988	0.989
fuel_system	0.995	0.991	0.917	0.989	0.978
coolant_temp_sensor	1.000	0.999	1.000	1.000	1.000
throttle_position_sensor	0.951	0.910	0.932	0.969	0.972
cold_start	0.995	1.000	1.000	1.000	1.000
Macro-F1	0.983	0.969	0.958	0.988	0.987
Test windows	12,568	21,802	30,336	19,270	10,454
Test sessions	13	47	39	12	18

Table 11: LOSO cross-validation summary over all 129 single-session folds.

Statistic	Macro-F1
Mean	0.917
Standard deviation	0.084
Best fold	1.000 (multiple sessions)
Worst fold	0.699 (live31)

Table 12: Withheld-configuration experiment: train on injector configuration A, evaluate on configuration B.

	Config A (train)	Config B (withheld)
air_system magnitude (kPa-eq.)	13.0	9.0
fuel_system magnitude (% LTFT)	18.0	24.0
coolant_temp_sensor target (°C)	42.0	38.0
throttle_position_sensor factor	1.35	1.50
Onset fraction	0.40	0.55
Ramp fraction	0.15	0.25
Noise σ	0.3	0.5
Macro-F1	0.892	0.859
Gap = 0.033 (same-config minus withheld-config)		